Computers & Operations Research 172 (2024) 106805

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

A cooperative coevolutionary genetic programming hyper-heuristic for multi-objective makespan and cost optimization in cloud workflow scheduling

Tomás Zaki [a], Yannik Zeiträg [b,*], Rui Neves [a], José Rui Figueira [b]

[a] Instituto de Telecomunicações, Instituto Superior Técnico, Universidade de Lisboa, Av. Rovisco Pais 1, Lisbon 1049-001, Portugal
[b] CEGIST, Instituto Superior Técnico, Universidade de Lisboa, Av. Rovisco Pais 1, Lisbon 1049-001, Portugal

ARTICLE INFO

Keywords:
Multi-objective optimization
Hyper-heuristic
Genetic programming
Cooperative coevolution
Cloud computing
Machine learning

ABSTRACT

This study presents a novel multi-objective approach for NP-hard workflow scheduling in cloud computing environments. Traditional rule-based heuristics offer flexibility but lack consistent superiority across all criteria and scenarios. The development of specific rules usually requires tedious manual refinement by experts. To overcome this limitation, our method leverages evolutionary computation and simulation to automatically derive new rules. Moreover, workflow scheduling involves two crucial and related aspects: task scheduling and the allocation of virtual machines. Our contribution includes three distinct algorithms: two that address each decision separately and a third approach that coevolves priority rules for both decisions simultaneously. Computational tests demonstrate superior performance, with an exploration of rules yielding a 72.91% larger hypervolume for optimizing makespan and costs compared to benchmark heuristics from the literature. The validation on unseen instances shows a 90.26% improvement in hypervolume performance, highlighting the robustness of our approach.

1. Introduction

As problems become more complex to solve and humans become more reliant on remote services for leisure and work, cloud computing's (CC) importance has become undeniable. Therefore, the efficiency and reliability of these services have become the target of important research.

In CC, schedulers are developed in order to optimize certain objectives, such as makespan, computational cost, resource utilization, and availability (Houssein et al., 2021). Many techniques have been used to optimize these objectives, such as heuristics and metaheuristics. However, a majority of approaches still use simple priority rules, such as *first-come-first-served* (Tsai et al., 2014). There are two main reasons these heuristics are used in place of other approaches. First, these rules are easy to implement. The second reason is that the rules are computationally efficient compared to other approaches, and little time is spent on constructing a solution. Therefore, they are particularly useful for dynamic environments, frequently encountered in real-world applications.

The multi-objective (MO) workflow scheduling in cloud computing (WSCC) problem is an NP-hard problem (Houssein et al., 2021). Workflows include a collection of tasks with interconnected relationships and dependencies, often represented as directed acyclic graphs (DAGs). A DAG is a directed graph with no cycles, meaning there is a defined direction for edges between nodes, and it is impossible to traverse from a node back to itself following those edges (Digitale et al., 2022). WSCC problems typically contain two interconnected scheduling decisions, one regarding the prioritization of waiting tasks, denoted as task scheduling, and one for assigning the selected task to an available virtual machine (VM), denoted as VM selection decision. With respect to the constructive heuristics, the priority rules are denoted as task scheduling rule (TSR) and VM selection rule (VMSR). Finding an optimal solution to this problem is far from trivial within a reasonable computational time, and most approaches mainly focus on optimizing a single metric rather than seeking to obtain a compromise solution between multiple objectives.

There are a couple of metrics to optimize when it comes to CC. An important one, that will be addressed in this study, is the makespan, the time required for a set of workflows to complete from start to finish. This aspect is important because, with the growing use of CC, these processes must also become more efficient and faster to ensure the future of these services. Another important objective of WSCC represents the costs caused by the execution of workflows. Services typically use operation expenditures for costs, i.e., cloud service users pay for

* Corresponding author.
E-mail address: yannik.zeitrag@tecnico.ulisboa.pt (Y. Zeiträg).

https://doi.org/10.1016/j.cor.2024.106805
Received 6 March 2024; Received in revised form 7 August 2024; Accepted 13 August 2024
Available online 20 August 2024
0305-0548/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

resources as they consume them. Therefore, another important metric when talking about CC is cost. Despite it being a cheaper alternative to capital expenditures where a customer pays for services upfront, the use of several CC resources can be costly, and therefore, optimizing costs is desired in this particular scenario (Rajasingh and Wesley, 2020). As there typically exists no solution that is superior in both criteria due to the competitive characteristics of both metrics, an appropriate compromise must be found. A possible approach is to scalarize multiple objectives to form a single objective function. However, the decision maker (DM) must be able to articulate their preferences before knowing about possible trade-offs. So-called *a posteriori* approaches on the other side enable the identification of multiple non-dominated solutions. The DM can obtain trade-offs and select the most appealing solution among the set of identified alternatives. In this study, we make use of the principles of one of the most well-known *a posteriori* metaheuristics, the NSGA-II (Deb et al., 2002). This method allows us to find compromise solutions along the Pareto front and to support the DM in identifying the most preferred one, if any.

As a recent field of study, most approaches have not yet been explored sufficiently by researchers. Most notably, hyper-heuristic (HH) approaches, aimed at automatically generating new heuristics combining concepts from machine learning and evolutionary computation, are underrepresented in the literature. HHs are high-level methodologies that automate the design of heuristics to solve computational problems. Instead of directly exploring the solution space, HHs typically employ evolutionary algorithms to navigate a space comprising heuristics or their components (Burke et al., 2003b). Proven by its efficiency in solving other scheduling problems, the study of genetic programming-based HH (GPHH) applied to the WSCC indicates a promising solution method with the potential of further improving upon a technology that has become quintessential to today's world. GP, as described by Koza (1992), merges evolutionary computation with automatic programming. This approach is well-suited for evolving individuals with different sizes and logical structures, such as the case with priority rules (Burke et al., 2009). Most existing GPHH approaches focus on evolving either the TSR or the VMSR in an isolated manner. There are only a few studies regarding the cooperative coevolution (COEV) of both rules (Xiao et al., 2019; Escott et al., 2023). In COEV, the process typically involves managing two distinct sub-populations, each evolving concurrently, which promotes mutual adaptation and refinement between the evolving rules (Yang et al., 2008). While an alternative approach via multiple trees exists for other scheduling problems, it has not yet been studied regarding WSCC. However, in studies of other scheduling problems, the COEV approach with two sub-populations has typically outperformed the multi-tree representation (Zhou et al., 2018). Nevertheless, what the existing COEV approaches for WSCC have in common is that they exclusively optimize on a single objective. To the best of the authors' knowledge, there is no study employing COEV within a GPHH framework for MO-WSCC problems. With this work, we intend to close the aforementioned gap by introducing a novel GPHH methodology that combines COEV and NSGA-II algorithm for coevolving non-dominated priority rules to efficiently solve WSCC problems.

To summarize, this study will contribute to WSCC in the following aspects:

- Genetic programming-based hyper-heuristic:
  The development of a novel approach to solve WSCC problems, making use of GPHH and simulation-based fitness evaluation.
- Cooperative coevolution of TSR and VMSR:
  The simultaneous generation of the two interconnected rules in order to prioritize tasks/VMs when constructing solutions in an integrated manner.
- Multiple conflicting objectives:
  The consideration of multiple conflicting objectives of WSCC problems, namely the makespan and costs.
- Decision support:
  The incorporation of the NSGA-II algorithm into the proposed GPHH method in order to enable the DM to select the most appropriate compromise solution *a posteriori*.

The remainder of this study will be organized in the following way. Section 2 is dedicated to a literature review of the existing approaches to WSCC. Section 3 describes the problem of WSCC and its various components. Section 4 provides an overview of the system used to approach this problem, including its GPHH algorithm and the simulation environment for the fitness evaluation. The experimental design is introduced in Section 5. Section 6 shows and discusses the results of the computational experiments. Finally, Section 7 concludes this study by elaborating on the system limitations and future works.

2. Literature review

This section presents a review of WSCC literature. In general, there are different approaches for solving WSCC problems. Based on the classification of a recent survey by Hosseini Shirvani (2023), we distinguish optimization algorithms into list scheduler, heuristics, metaheuristics, hybrid heuristics, and hyper-heuristics. An overview of the proposed classification can be found in Fig. 1, while the boxes highlighted in dark gray represent the context of this study. A more thorough review of WSCC solving methods can be seen in the survey by Masdari et al. (2016).

2.1. List schedulers, heuristics, metaheuristics, and hybrid heuristics

Starting with the simplest class of solution methods for WSCC, list schedulers typically utilize priority rules to iteratively construct solutions (Madni et al., 2017). These schedulers directly assign priorities to tasks based on predefined criteria. These priorities are then used to construct a list of tasks to be scheduled, and the scheduler selects tasks for execution based on the order in this list. Simple priority rules include first-come-first-serve (FCFS) (Rimal and Maier, 2017), minimum completion time (MCT) (Rimal and Maier, 2017), min-min (Patel et al., 2015), and max–min (Devipriya and Ramesh, 2013). However, while efficient and straightforward to implement, simple priority rules often only achieve close-to-optimal solutions. Another class of list schedulers considers more complex criteria, including task dependencies, when calculating priorities. Prominent examples include heterogeneous earliest finish time (HEFT) (Topcuoglu et al., 2002) and predictable earliest finish time (PEFT) (Arabnejad and Barbosa, 2013). Hybrid list scheduler algorithms can be found in studies by Shirvani and Talouki (2021) and NoorianTalouki et al. (2022). Hosseini Shirvani and Noorian Talouki (2022) address the challenge of WSCC on hybrid cloud architectures, considering task interdependencies, heterogeneity, and cost diversity. The authors introduce a novel hybrid bi-objective optimization approach, which minimizes both makespan and monetary costs by integrating simulated annealing and task duplication algorithms.

Heuristic algorithms for WSCC can be broadly categorized into clustering, duplication, and replication techniques (Hosseini Shirvani, 2023; Wu et al., 2015). In the clustering technique, tasks that are dependent on each other are grouped into clusters, with each cluster assigned to a single processor. This approach significantly reduces communication costs between tasks within the same cluster since data transfers between processors are minimized. However, overusing clustering can limit parallel execution, effectively reducing the performance to that of a serial execution. Therefore, it is essential to determine an optimal clustering threshold to balance communication overhead and parallelism. The duplication technique focuses on identifying critical tasks that have multiple dependent tasks and duplicating these tasks across different processors. This reduces waiting times for successor tasks and enhances parallelism, leading to a shorter overall makespan.

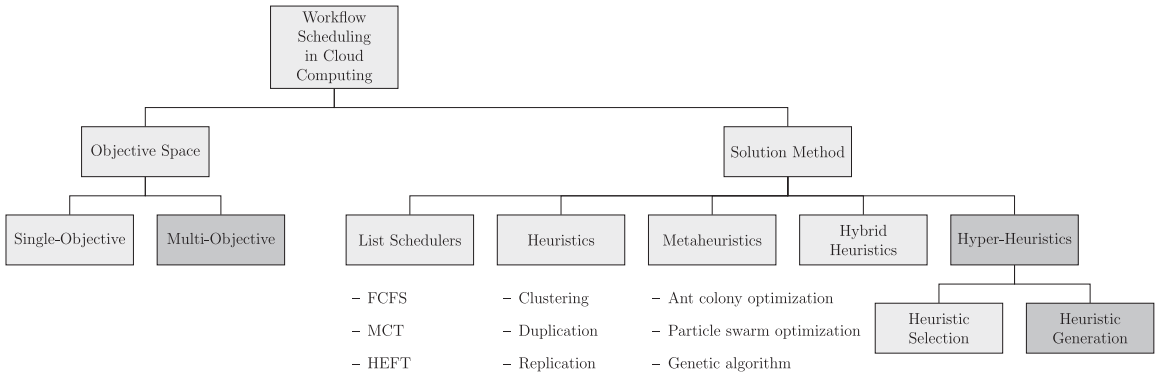


Fig. 1. Classification of the literature.

This approach is particularly useful when immediate successor tasks need to start execution without delay. Lastly, the replication technique is designed to improve scheduling performance in workflows that require continuous data streaming. By replicating data in a pipeline or interleaved manner, this method ensures that data is readily available, reducing idle times and improving throughput. This technique is especially advantageous in scenarios where constant data flow between processors is crucial. For more details, the interested reader is referred to Lin et al. (2013)

A more complex but usually more effective method represents metaheuristic approaches. Among many, the most frequently studied approaches are nature-inspired algorithms such as ant colony optimization, particle swarm optimization, and genetic algorithm. A comparative study of the aforementioned metaheuristic approaches can be found in the works by Kalra and Singh (2015) and Singh et al. (2021). Bhardwaj et al. (2020) further compare the performance of several metaheuristics to simple heuristics. With respect to multiple objectives, NSGA-III has been studied by Imene et al. (2022). Zade et al. (2021) address the challenge of ensuring the critical quality of service in cloud data centers for extensive data-centric tasks by introducing a MO fuzzy-based task scheduling algorithm. The authors employ the squirrel search algorithm and parallel squirrels search algorithm to optimize energy cost, makespan, degree of imbalance, and security levels simultaneously. Reddy and Reddy (2023) present a MO optimization framework for cloud workflow scheduling. With the aim of optimizing energy consumption, makespan, and cost for the heterogeneous computing scheduling problem, the authors utilize a combination of metaheuristics. Xie et al. (2023) introduce a two-stage preference-driven MO evolutionary algorithm to investigate the complexity of multi-criteria workflow scheduling in the cloud. Formulating the problem as an MO optimization problem with user-specific preferences, the algorithm employs a preference distance strategy in the first stage to expedite solutions around the DMs' region of interest and a preference region ranking strategy in the second stage to focus on exploration. Chiang et al. (2023) propose a balanced candidate suffrage value algorithm for efficient task scheduling in CC environments under consideration of the load balancing among working nodes. In essence, the authors utilize different criteria such as the smallest suffrage value for task prioritizing to achieve better load balance and makespan across different heterogeneous network environments. Shukri et al. (2021) present an enhanced version of their previously introduced multi-verse optimizer as an advanced task scheduler, comparing it with the original multi-verse optimizer and particle swarm optimization.

In order to further improve the performance, a typical approach is to hybridize existing metaheuristics by combining them with other metaheuristics and/or heuristics to take advantage of different algorithm's strengths. In Xiong et al. (2017), a study was performed combining genetic algorithm with johnson's rule. A combination of a particle swarm optimization and a moth search algorithm with differential evolution is proposed in Abd Elaziz et al. (2019). The use of two hybrid heuristics based on particle swarm optimization and other heuristics is proposed in Alsaidy et al. (2022). Another noteworthy hybridized algorithm to solve a task scheduling problem in CC is provided by Gawali and Shinde (2018), which combines several algorithms, such as the BATs algorithm and the modified analytic hierarchy process. Zhang et al. (2023) address the challenges of real-time workflow scheduling in dynamic and uncertain cloud environments by proposing a hybrid method, which combines genetic algorithm and deep reinforcement learning for an MO approach. Their algorithm divides the scheduling process into two stages, utilizing genetic algorithm for global search capability in calculating the execution scheme and reinforcement learning for environment-aware decision-making to assign workflows to suitable VMs.

2.2. Hyper-heuristics

Besides the above-described approaches, there is another category of methods, the so-called HH, which will be the focus of this study. These algorithms use the concepts of evolutionary algorithms and machine learning to generate or select heuristics for a class of problems, rather than solving a concrete problem. The main differences to the previously introduced methods rely on the search space of the algorithm. Instead of operating on the solution space directly, HH methods search on the space of heuristics (Burke et al., 2003b). Therefore, HH algorithms do not provide solutions directly. Instead, they generate new heuristics that can then be applied to solve problems similar to the ones on which the heuristics have been trained. In the literature, there are two approaches to distinguish (Burke et al., 2010):

- Heuristic Selection
  The first class of HH methods is called heuristic selection, where an algorithm is employed to select from a set of predefined heuristics or metaheuristics. For a detailed overview of existing literature regarding heuristic selection methods in the scheduling domain, the interested reader is referred to the survey by Drake et al. (2020).
- Heuristic Generation
  The second category describes heuristic generation methods, where usually a genetic programming (GP) algorithm is used to generate new heuristics from predefined components, such as problem-describing parameters. As a specialized variant of genetic algorithms, GP integrates evolutionary computation and automatic programming, offering a powerful means to evolve mathematical expressions, including priority rules (Koza, 1992). Unlike many other machine learning methods, GP possesses the unique ability to learn not only the parameters but also the underlying structure of the rule. This flexibility extends to the inclusion of any operators, enabling customization for specific problem

domains. Additionally, the symbolic models produced by GP demonstrate computational efficiency, outperforming ensemble models such as random forests or gradient boosting. Over the past two decades, numerous GPHH approaches have been proposed for automatically designing heuristics across various scheduling problems. Interested readers are encouraged to explore comprehensive surveys by Branke et al. (2015) and Nguyen et al. (2017) for further insights into these advancements.

2.2.1. Hyper-heuristics in cloud computing

With respect to CC, Tsai et al. (2014) presented an early HH approach employing a genetic algorithm to perform heuristic selection in a perturbation nature of the search space. Their experiments are performed in different simulation environments with several workflows and tasks. They conclude from their gathered data that their proposed solution outperforms other metaheuristic approaches in terms of makespan. This work demonstrates the use of HH for task scheduling in CC since it takes advantage of the simplicity of low-level heuristics while being able to produce more complex and better-suited solutions that are not as computationally expensive as metaheuristic approaches.

Recently, a study performed by Yu et al. (2019) was made regarding the use of HH in CC. Their motivation stemmed from the limitations of techniques like HEFT, which struggle with multiple objectives, diverse VM types, and task dependencies within workflows. They compare their GPHH solution to several heuristic and metaheuristic solutions such as particle swarm optimization, HEFT, round robin, weighted round robin, and a random algorithm. To account for the complexity and heterogeneity of the cloud environment, their experiments propose the use of different datasets, with different kinds of tasks and task sizes, but also using different kinds of VMs. Their results show how their algorithm outperforms all others in various metrics due to how flexible their solution is in a problem whose nature is dynamic and complex. Yu et al. (2021) extended their GPHH approach to consider multiple objectives. With the aim of optimizing the makespan and costs, the GPHH algorithm employs genetic operators to evolve and select promising heuristic strategies, ensuring a robust and adaptable scheduling solution. In their latest development, Yu et al. (2022) tackle the complexities of scheduling tasks of dynamically arriving workflows within cloud environments. Experimental evaluations using benchmark datasets demonstrate the effectiveness of the GPHH approach, outperforming traditional heuristics and achieving superior performance in terms of hypervolume. Alkhanak and Lee (2018) introduce the completion time-driven HH approach for cost optimization of scientific WSCC. The proposed method employs four well-known population-based metaheuristic algorithms, acting as low-level heuristics, to schedule workflow tasks on cloud resources. The proposed approach dynamically chooses the most suitable low-level heuristics by considering the best-computed workflow completion time. Experimental results on real-world datasets demonstrate that the proposed method significantly outperforms baseline approaches in terms of cost optimization and execution time. Although the proposed method optimizes the completion time, the authors claim to indirectly optimize costs due to a correlation between the two objectives. Xiao et al. (2019) propose a COEV GP framework for the WSCC problem. Their approach algorithm simultaneously coevolves TSRs and VMSRs, ensuring that they are complementary and work together effectively. Experimental results on benchmark datasets demonstrate that the COEV GP framework outperforms traditional metaheuristics and hyper-heuristics in terms of makespan, resource utilization, and fairness. Escott et al. (2020) propose a GPHH approach for dynamic WSCC, addressing the challenge of efficiently allocating tasks to VMs in the cloud while accommodating dynamic workflow arrivals. Their approach automatically evolves and optimizes scheduling heuristics, demonstrating superior performance compared to existing heuristics in terms of makespan and resource utilization. Escott et al. (2023) extended their previous work by introducing a COEV GPHH for budget-constrained dynamic multi-workflow scheduling in CC. This approach introduced two types of heuristic rules: VM selection rule and budget alert rule, enabling efficient resource allocation while adhering to budget constraints. The COEV GP algorithm, utilizing multiple populations to evolve different rules that consider both decisions, further enhanced the scheduling optimization. Their evaluation on a benchmark dataset demonstrated the effectiveness of the COEV GP hyper-heuristic, outperforming existing heuristics while considering budget constraints. Xu et al. (2023) introduce a multi-tree GP approach for dynamic workflow scheduling in fog computing. The proposed method employs a multi-tree representation to handle the routing and sequencing decision points simultaneously. This representation allows the algorithm to evolve scheduling policies that effectively utilize the resources of mobile devices, edge servers, and cloud servers. Experimental results on benchmark datasets demonstrate that the proposed approach achieves significant improvements in makespan compared to existing state-of-the-art methods, particularly under dynamic workload conditions.

2.2.2. Hyper-heuristics in other scheduling domains

Besides CC, one of the most explored scheduling problems with respect to GPHH methods represents the production scheduling problem. Similar to the WSCC, a common constructive heuristic uses priority rules to assign jobs/tasks to machines and schedule their sequence of processing. In the early stages, initial approaches to automatically evolve priority rules through GPHH considered simple shop configurations such as single-machine scheduling (Dimopoulos and Zalzala, 2001) and parallel-machine scheduling (Jakobović et al., 2007). Ozsoydan and Sağir (2021) extended the scope to a flow shop scheduling environment. Addressing multiple objectives, Nguyen et al. (2013) applied an extension of the NSGA-II to obtain non-dominated solutions. Notably, initial endeavors in evolving priority rules for scheduling decisions assumed a fixed priority rule for machine assignment (Ho and Tay, 2005; Tay and Ho, 2008). Subsequent advancements in GPHH techniques have progressed towards concurrently evolving rules for both decisions. Most studied methods include GP with two subpopulations or GP with two sub-trees, as demonstrated by Zhou et al. (2020) and Zeiträg et al. (2024). Several alternative approaches tailored to diverse specifications have been devised, addressing various aspects such as extended technical precedence constraints (Fan et al., 2021), decision-making support (Zeiträg and Figueira, 2023), and the incorporation of surrogate-assisted fitness evaluation (Gil-Gala et al., 2023; Zeiträg et al., 2022). GPHH methods have also been successfully implemented on other planning and scheduling problems with related characteristics to the WSCC such as project scheduling problems (Lin et al., 2020; Chen et al., 2021; Luo et al., 2022) and lot sizing problems (Hein et al., 2018).

2.3. Summary and gap

Numerous optimization algorithms have been developed for WSCC over the past decades. While metaheuristics and hybrid heuristics typically provide better solutions, simple heuristics convince practitioners through their simplicity and fast responsiveness. Particularly in dynamic environments where tasks randomly arrive in the system, quick solution generation is inevitable for real-world applications. Although there exist various simple heuristics that have been manually developed by experts, machine-learning-based methods to automatically learn new heuristics have barely been studied yet. Some authors explicitly underlined the investigation of such methods as a promising extension to existing approaches. Another aspect that has not yet been sufficiently studied with respect to simple heuristic methods for WSCC is the consideration of multiple conflicting objectives. To this end, DMs should be supported in finding solutions that generate an appropriate compromise. Among others, GPHH has proven to be a suitable method to automatically evolve new high-performing rule-based heuristics for scheduling problems. Table 1 summarizes the reviewed literature that

uses GPHH for solving WSCC problems. Note, that the first two works listed in the table do not use GP, they rather employ genetic algorithms in an HH context for dynamically selecting an appropriate metaheuristic.

While existing literature showcases MO-GPHH approaches, these primarily address the two interconnected scheduling decisions in an isolated manner. Additionally, there is a body of work on COEV approaches that simultaneously consider both scheduling decisions for single-objective WSCC problems. However, a notable gap persists in the literature. To the best of the authors' knowledge, there is a dearth of studies that seamlessly integrate multiple objectives into a COEV algorithm for tackling the complexities inherent in the WSCC problem. This research gap underlines the need for an innovative approach that unites the benefits of MO optimization with the power of COEV, providing a more comprehensive solution to the challenges posed by WSCC scenarios. This work's focus is to close the aforementioned gaps by developing a novel MO optimization approach that incorporates a COEV simulation-based GPHH method to automatically learn non-dominated priority rules.

3. Problem formulation

This study addresses the NP-hard multi-workflow multi-processor scheduling problem in CC, focusing on the efficient allocation of tasks within workflows on a designated set of machines. Workflows are modeled as DAGs, denoted as $W = (T, D)$, where T represents a set of n tasks ($T = \{T_1, T_2, \dots, T_i, \dots, T_n\}$), and D represents a set of m dependencies ($D = \{D_1, D_2, \dots, D_q \dots, D_\ell\}$). Each dependency ($D_k = (T_i, T_j)$) indicates that task T_j directly succeeds task T_i, establishing a sequential relationship within the workflow. The computational effort *per* task is assumed to be known, expressed by the number of machine instructions needed for its execution

The problem at hand involves scheduling p workloads ($W = \{W_1, W_2, \dots, W_o \dots, W_p\}$) on a set of m VMs ($VM = \{VM_1, VM_2, \dots, VM_k \dots, VM_m\}$). The problem encompasses two crucial decisions. The first decision involves task scheduling, where tasks are prioritized for execution. The second decision is regarding resource allocation, where tasks are assigned to VMs. The overall aim is to minimize makespan and costs, ensuring an efficient and balanced execution of tasks while navigating the intricate dependencies within and across workflows.

For this study, the following assumptions will be made:

1. No preemption is allowed, meaning that the machine must complete a task once it is assigned.
2. The scheduling of tasks is dynamic.

The dynamic nature of the problem refers to the scenario where workflows arrive randomly in the cloud environment, and the characteristics of these tasks are not known to the scheduler until they appear in the environment. In other words, the scheduler does not know when tasks will arrive and how they are specified, such as processing time or resource requirements. Therefore, tasks must be assigned to machines and scheduled based on real-time information as they are encountered in the cloud environment.

3.1. Model notation

The sets, indices, parameters, and variables for the mathematical formulation of the problem under consideration are denoted as follows:

Indices and sets:

- i: Index of tasks, $i = 1, \dots, n$
- k: Index of VMs, $k = 1, \dots, m$
- C_k: Set of tasks assigned to VM k, $k = 1, \dots, m$

Parameters:

- t_i^p: Processing time of task i
- r_i: Release time of task i
- c_k: The cost of running VM k
- M: A large positive number

Variables:

- $t_i^s \geqslant 0$: Start time of task i
- $t_i^c \geqslant 0$: Completion time of task i
- $t_k^{run} \geqslant 0$: Runtime of VM k

Binary variable:

- $y_{i'ik} \in \{0, 1\}$: $y_{i'ik} = 1$ if task i' precedes task i on VM k, 0 otherwise

3.2. Performance measures

For the subsequent study, we selected two of the most relevant objectives for WSCC optimization problems, namely the makespan and the costs. Their definitions and their mathematical expressions are introduced in the following.

1. *Makespan.* The first objective is to minimize the makespan. Makespan is defined as the time VMs take to complete all tasks of all workflows (Murad et al., 2022). To this end, the makespan can be described as the timespan between the start and the completion of a sequence of tasks.

$$\min f_1(t^c) = \max\{t_i^c | i = 1 \dots, n\} \tag{1}$$

2. *Cost.* The second objective is the minimization of costs. The use of CC comes with an operational cost. The value depends on the number of resources deployed and the characteristics of these resources (Rajasingh and Wesley, 2020). For this work, the focus will be on the costs of deploying and running VMs. Since most cloud services charge costs on an hourly basis, we use the rounded-up value of the runtime.

$$\min f_2(t^{run}) = \sum_{j=1}^{m} \lceil t_j^{run} \rceil \times c_j \tag{2}$$

3.3. Mathematical programming model

The mathematical programming model of the WSCC problem can be formulated as follows:

$$\min f_1(t^c) \tag{3}$$

$$\min f_2(t^{run}) \tag{4}$$

s.t. :

$$s_i \geqslant r_i, i = 1, \dots, n \tag{5}$$

$$t_i^c \geqslant t_i^s + t_i^p, i = 1, \dots, n \tag{6}$$

$$t_{i'}^s \geqslant t_i^c - y_{i'ik} M, i, i' = 1, \dots, n, i < i', k = 1, \dots, m \tag{7}$$

$$t_i^s \geqslant t_{i'}^c - (1 - y_{i'ik}) M, i, i' = 1, \dots, n, i < i', k = 1, \dots, m \tag{8}$$

$$y_{i'ik} + y_{ii'k} = 1, i, i', \dots, n, i < i', k = 1, \dots, m \tag{9}$$

$$t_k^{run} \geqslant \sum_{i \in C_k} t_i^p, k = 1, \dots, m \tag{10}$$

$$t_k^{run} \geqslant 0, k = 1, \dots, m \tag{11}$$

$$t_i^s \geqslant 0, i = 1, \dots, n \tag{12}$$

Table 1
Classification of the reviewed HH literature for WSCC.

Reference	Heuristic space				Hyper-heuristic		Objective space		MO method[a]		Objective	
	Task scheduling rule (TSR)	VM scheduling rule (VMSR)	Coevolution (TSR & VMSR)	Metaheuristics	Heuristic selection	Heuristic generation	Single objective	Multi-objective	*a priori*	*a posteriori*	Makespan	Costs
Tsai et al. (2014)				x	x		x				x	
Alkhanak and Lee (2018)				x	x		x				x	
Yu et al. (2019)		x				x		x	x		x	x
Xiao et al. (2019)			x			x	x				x	
Escott et al. (2020)		x				x	x				x	
Yu et al. (2021)		x				x		x		x	x	x
Yu et al. (2022)		x				x		x		x	x	x
Escott et al. (2023)			x			x	x				x	
Xu et al. (2023)	x	x				x	x				x	
This work	**x**	**x**	**x**			**x**		**x**		**x**	**x**	**x**

[a] Only applicable for multi-objective studies

Table 2
Terminal and function set.

Type	Symbol	Description
Terminals TSR	TL	Task's length (in MI)
	TNC	Task's number of children tasks
	TWT	Task's waiting time
	TACT	Task's average computational time
	WD	Workflow's depth
	NRT	Number of ready tasks
Terminals VMSR	TL	Task's length
	TNC	Task's number of children tasks
	TWT	Task's waiting time
	VMM	VM's speed (in MIPS)
	VMC	VM's cost (*per* hour)
	EFT	Earliest finish time
Functions TSR & VMSR	ADD	Addition
	SUB	Subtraction
	MUL	Multiplication
	DIV	Protected division (DIV$(a,b)=1$, if $b=0$)
	MAX	Maximum
	MIN	Minimum

$$t_i^c \geqslant 0, i = 1, \ldots, n \tag{13}$$

$$y_{i'ik} \in \{0,1\}, i, i' = 1, \ldots, n, i \neq i', k = 1, \ldots, m \tag{14}$$

Eqs. (3) and (4) represent the objectives of the problem under investigation. Eq. (5) guarantees that a task starts after its attribution to a VM. Eq. (6) ensures that the difference between the completion time and the starting time of a task is greater or equal to its processing time.Eq. (7), Eq. (8), and Eq. (9) define that one machine can only process one task at a time and that the sequence of tasks is respected according to the order that is specified by their dependencies. Eq. (10) computes the runtime of each machine. Eq. (11), Eq. (12), and Eq. (13) formulate the non-negativity of the continuous decision variables. Eq. (14) defines $y_{i'ij}$ as binary variable.

4. Methodology

In this section, we first introduce an overview of the system architecture of the proposed MO-GPHH approach to solve WSCC problems. The two main modules, the genetic programming and the simulation environment will be further discussed in detail in the subsequent parts of this section.

4.1. System architecture

The proposed approach, implemented in Java, contains two interconnected frameworks, one responsible for the genetic programming implementation and another one for modeling and simulating the WSCC problem. Fig. 2 presents an overview of the system architecture. The details of both components will be introduced in the following subsections.

4.2. Genetic programming

For the implementation of the MO-GPHH algorithm, we used a well-established evolutionary computation framework written in Java (ECJ). ECJ provides essential functions for the implementation of the genetic programming algorithm and MO evolutionary operators such as the non-dominated sorting. The proposed MO-GPHH algorithm is composed of five main modules:

- Population Initializer
  The first step is to initialize the individuals of the first generation. An individual is represented as a GP tree (see Fig. 3). These trees contain a scheduling heuristic, with the intermediate nodes representing arithmetic functions and terminal nodes representing the most commonly used properties of workflows and VMs. Table 2 shows an overview of the terminals and functions, used to represent nodes in the GP tree. Please note, that since WSCC problems contain two crucial scheduling decisions, we propose to study to evolve priority rules regarding task scheduling and VM allocation, denoted as task scheduling rule (TSR) and VM scheduling rule (VMSR). The task's length is defined in million instructions (MI). The workflow's depth is defined as the furthest path from the root task to the final task(s) to complete the workflow.
  The initial population is generated utilizing the ramped half-and-half method, following the procedure outlined by Burke et al. (2003a). This algorithm begins by selecting a maximum depth range for the trees. Subsequently, with a 50% probability, it employs either the grow algorithm or the full algorithm. Both algorithms operate recursively to generate trees, with the grow algorithm yielding trees with variable branch depths (up to the maximum initial depth), and the full algorithm constructing trees with branches extending to the maximum depth. Any duplicated individuals generated by these methods are systematically removed from the population.
- Fitness Evaluator:
  The next step is the fitness evaluation of individuals, which will be performed in the simulation environment. To this end, an individual is passed to the simulation module, in which the individual is employed to construct schedules for a set of training instances (see Section 4.3). The average performance of the schedules' performance (makespan and costs) is returned to the fitness evaluator module. This step's goal is to gain information on the fitness of individuals so that they can be discarded or kept, depending on their performance.

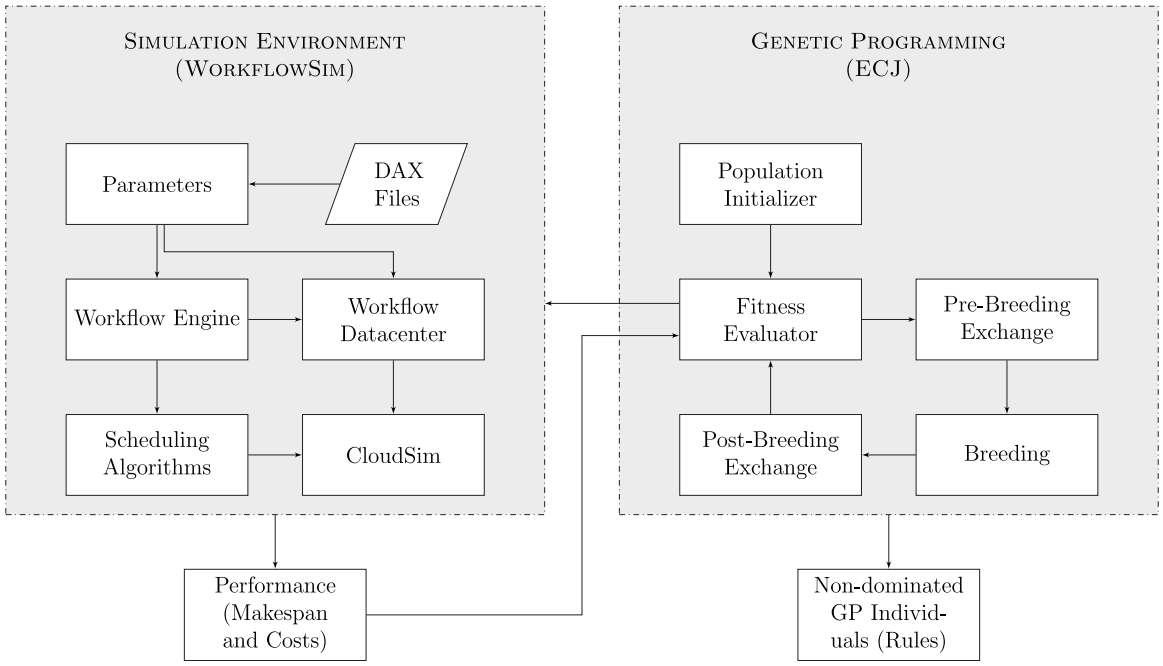


Fig. 2. Proposed System Overview.

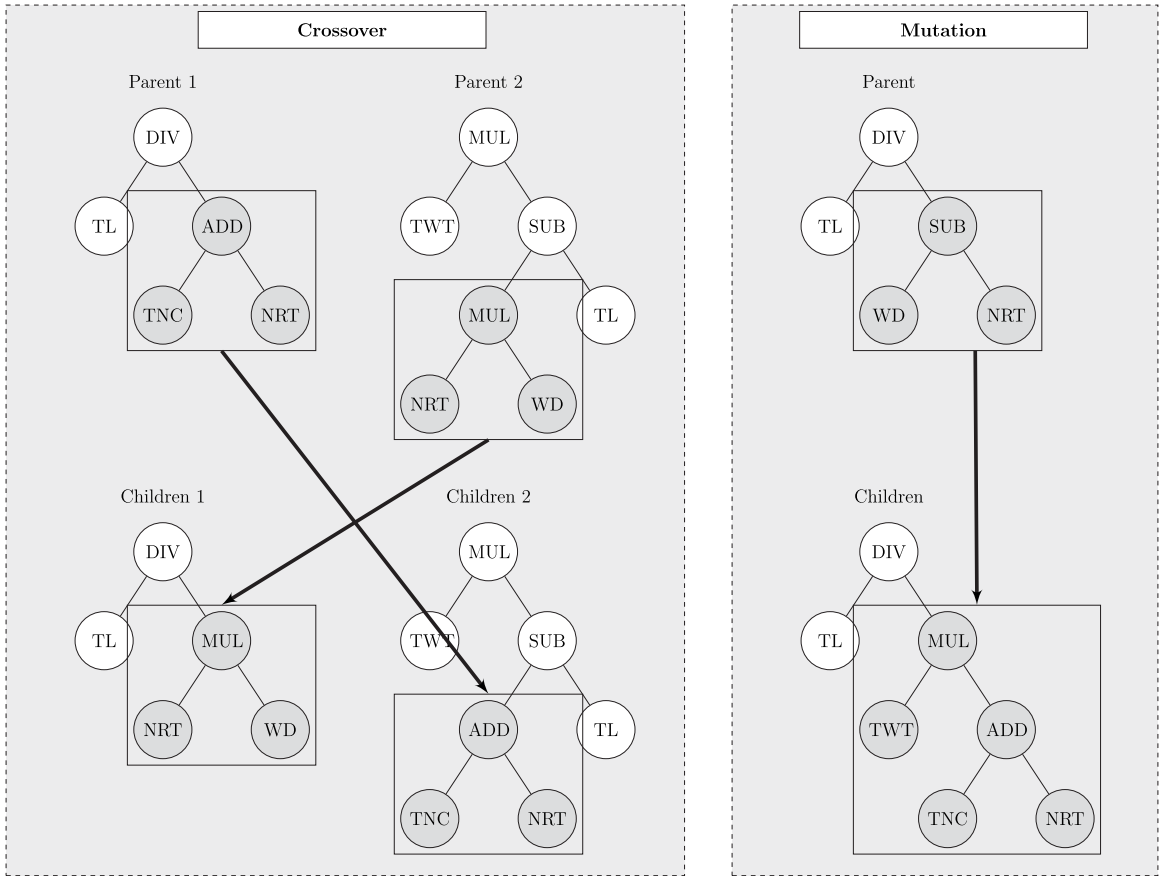


Fig. 3. Crossover (left) and Mutation (right) Operators.

– Pre-Breeding Exchange:
From the evaluated individuals, the NSGA-II algorithm uses the non-dominated sorting approach to select the best-performing ones for reproduction based on the non-dominated rank and the crowding distance (Deb et al., 2002).

- Breeding:
  The following evolutionary operators are used in the proposed MO-GPHH algorithm to guide the algorithm into a promising objective space:
  - Crossover:
    Crossover, or recombination, is the operator that uses the genetic information of two parents to generate offspring. The branches of two parent individuals are randomly selected and exchanged in order to create two new offspring.
  - Mutation:
    Mutation is the genetic operator that guarantees genetic diversity in a pool of individuals. To achieve this, the operator changes the subtree of the parent individual of the previous generation to generate a new offspring.

  Fig. 3 illustrates both the process of crossover, with each parent passing a portion of their GP tree to the offspring, and the process of mutation, where the offspring's subtree has a variation of nodes compared to the parent as a result of this process.
- Post-Breeding Exchange:
  After the reproduction phase, the initial population is updated with the newly generated individuals. Duplicate individuals are being removed as described in the initialization phase.

The last four modules are iteratively executed until a stopping criterion is met. The outcome of the method represents the non-dominated individuals of the final population. Each individual consists of a priority rule or a combination of rules that can be used for constructing schedules for upcoming problems of a similar type. Note, that the entire MO-GPHH procedure represents the training phase with the aim of generating better-performing heuristics. The DM can finally select the most preferred alternative from the evolved set of trade-off heuristics, if any. Algorithm 1 presents the pseudocode of the proposed MO-GPHH.

Algorithm 1 Pseudocode for the proposed MO-GPHH for WSCC problems.

Input: A set of training instances $S = \{S_1, S_2, \dots, S_k\}$;
Output: A set of non-dominated solutions P^*;
Initialize population $P^\mu \leftarrow \{I_1, I_2, \dots, I_{popsize}\}$;
$gen \leftarrow 0$;
while $(gen \leq gen_{max})$ **do**
  Remove duplicates from P^μ;
  foreach $(I_i \in P^\mu)$ **do**
    foreach $(S_k \in S)$ **do**
      $F(S_k) = \text{SimulationAlgorithm}(S_k, I_i)$; ▷ see Section 4.3
    end for
    Compute the average fitness across all training instances;
    Assign the average fitness to the individual I_i;
  end for
  Assign ranks and crowding distance for all individuals of P^μ based on their fitness;
  $P^\mu \leftarrow$ select(P^μ);
  $P^\lambda \leftarrow$ apply crossover & mutation to P^μ;
  Remove duplicates from P^λ;
  Assign ranks and crowding distance for all individuals of P^λ;
  $P^\mu \leftarrow P^\mu \cup P^\lambda$;
  $gen \leftarrow gen + 1$;
end while
$P^* \leftarrow$ first front of P^μ;
Return P^*.

4.3. Simulation environment

In order to simulate the cloud environment, WorkflowSim (Chen and Deelman, 2012), a Java-based extension of the simulation framework CloudSim (Calheiros et al., 2009) was used as the basis for our implementation. The most noteworthy modules of WorkflowSim are briefly explained in the following:

- Parameters: This module is responsible for parameter configuration such as workflows, the number of VMs, and the scheduling algorithm.
- WorkFlow Datacenter: A module that supplies resources for the VMs to process the tasks and define the cost model.
- WorkFlow Engine: A class that manages tasks' and VMs' life cycles.
- CloudSim: A modeling and simulation framework for CC infrastructures and services that is responsible for the actual simulation runs.
- Scheduling Algorithms: A directory containing all scheduling algorithms that are called by the system. The incoming rules for task and VM prioritization are processed by these algorithms.
- DAX Files: This directory contains DAX (directed acyclic graph in XML) files, which are XML-based files representing workflows. These files specify task dependencies, aiding cloud platforms in scheduling and executing tasks across distributed resources. They also include metadata for optimized task allocation, such as resource requirements and execution constraints.

The output of the simulation environment module represents the performance of the individual tested. The performance contains the makespan of the scheduled tasks and the costs of running VMs. We propose three different MO-GPHH algorithms. As described in Section 3, the WSCC problem contains two prioritization decisions which take place in a hierarchical manner. To construct a schedule, first, a task needs to be selected from the waiting tasks. In the second step, an appropriate VM is to be chosen for the previously picked task. Inspired by the HEFT algorithm (Topcuoglu et al., 2002), we propose one algorithm that operates on each decision independently and another algorithm that considers both interconnected decisions in an integrated manner. The latter represents the main contribution to the MO-GPHH literature regarding WSCC problems. Details of the proposed scheduling algorithms are discussed in the following:

- Task scheduling algorithm:
  The task scheduling algorithm focuses on optimizing the prioritization of tasks. While the HEFT algorithm ranks the tasks based on their computational costs, we employ our MO-GPHH framework for automatically evolving new priority rules. The subsequent decision regarding VM allocation is not subject to this algorithm. A task is assigned to a VM based on its earliest finish time (EFT) as proposed in the original heuristic (Topcuoglu et al., 2002). The input of the task scheduling algorithm represents the scenario and the individual from the GPHH module, which contains the TSR. This rule is employed by the algorithm to construct a schedule and compute the performance measures, the makespan and the costs. These values are returned by the algorithm and sent back to the GPHH module (see Fig. 2). The pseudocode of the task scheduling algorithm is shown in Algorithm 2.
- VM scheduling algorithm:
  The VM scheduling algorithm focuses on optimizing the prioritization of VMs for the assignment of the previously selected task. In this approach, the tasks are first ranked based on their computational costs, as proposed by the original HEFT algorithm (Topcuoglu et al., 2002). The second decision regarding the VM allocation is where our MO-GPHH framework aims to iteratively find better-performing VMSRs. Besides the training scenario, the algorithm required the VMSR as input. The output of the algorithm represents the performance measures, the makespan and the costs, of the constructed schedule. These values are returned to the fitness evaluator module of the GPHH system (see Fig. 2). The pseudocode of the VM scheduling algorithm is presented in Algorithm 3.

Algorithm 2 Pseudocode for the task scheduling algorithm.

Input: scenario S_k, individual I_i; ▷ task scheduling rule (TSR)
Output: makespan and costs of the constructed schedule;
Initialize the set of VMs $VM = \{VM_1, VM_2, \ldots, VM_m\}$;
Initialize the set of unscheduled tasks $T = \{T_1, T_2, \ldots, T_n\}$;
while (unscheduled tasks) **do**
  Compute the unscheduled tasks' priority based on the TSR; ▷ TSR from I_i
  Rank unscheduled tasks based on the priority in decreasing order;
  foreach ($T_i \in T$) **do**
    foreach ($VM_k \in VM$) **do**
      Compute the VM's priority based on the earliest finish time (EFT) for T_i on VM_k;
    end for
    Assign T_i to the VM with the highest priority;
  end for
end while
Calculate makespan and costs;
Return makespan and costs;

Algorithm 3 Pseudocode for the VM scheduling algorithm.

Input: scenario S_k, individual I_i; ▷ VM scheduling rule (VMSR)
Output: makespan and costs of the constructed schedule;
Initialize the set of VMs $VM = \{VM_1, VM_2, \ldots, VM_m\}$;
Initialize the set of unscheduled tasks $T = \{T_1, T_2, \ldots, T_n\}$;
while (unscheduled tasks) **do**
  Compute the unscheduled tasks' priority based on the computational costs;
  Rank unscheduled tasks based on the priority in decreasing order;
  foreach ($T_i \in T$) **do**
    foreach ($VM_k \in VM$) **do**
      Compute the VM's priority based on the VMSR for T_i on VM_k; ▷ VMSR from I_i
    end for
    Assign T_i to the VM with the highest priority;
  end for
end while
Calculate makespan and costs;
Return makespan and costs;

– Integrated scheduling algorithm:
The integrated scheduling algorithm is designed to consider both scheduling decisions simultaneously. To this end, our MO-GPHH framework operates on the priority rules for both decisions, regarding task scheduling and VM allocation. GP is able to simultaneously coevolve TSRs and VMSRs making use of two subpopulations, one for each decision, denoted as COEV. Compared to the two above-introduced approaches, each individual of the MO-GPHH algorithm at hand contains two rules (TSR and VMSR), a collaboration of rules from each sub-population (see Fig. 4). The combination of both rules will be denoted as a workflow scheduling policy (WSP). The MO-GPHH algorithm aims to coevolve rules for both scheduling decisions. The input of the integrated scheduling algorithm represents the training scenario and the individual containing the TSR and VMSR. These rules and then employed in order to construct a schedule and compute the performance measures, the makespan and the costs. These values are then returned to the GP module (see Fig. 2). The pseudocode of the integrated scheduling algorithm is shown in Algorithm 4.

Algorithm 4 Pseudocode for the integrated scheduling algorithm.

Input: scenario S_k, individual I_i; ▷ TSR and VMSR
Output: makespan and costs of the constructed schedule;
Initialize the set of VMs $VM = \{VM_1, VM_2, \ldots, VM_m\}$;
Initialize the set of unscheduled tasks $T = \{T_1, T_2, \ldots, T_n\}$;
while (unscheduled tasks) **do**
  Compute the unscheduled tasks' priority based on the TSR; ▷ TSR from I_i
  Rank unscheduled tasks based on the priority in decreasing order;
  foreach ($T_i \in T$) **do**
    foreach ($VM_k \in VM$) **do**
      Compute the VM's priority based on the VMSR for T_i on VM_k; ▷ VMSR from I_i
    end for
    Assign T_i to the VM with the highest priority;
  end for
end while
Calculate makespan and costs;
Return makespan and costs;

5. Experimental design

The following section will provide an overview of the workflows, instances, GPHH configurations, benchmark algorithms, and performance metrics used for this study.

5.1. Workflows

Workflows are a set of tasks that are interconnected and have interdependencies. They are typically represented as DAGs and codified in xml files. In the literature, there exist a number of established benchmark workflows[1] based on real-world applications such as Cybershake, Epigenomics, Inspiral, Montage, and Sipht (Bharathi et al., 2008; Juve et al., 2013). All workflows have two different main features. These include the length and size of a task and the dependencies between them. They exist in different sizes from 24 to 1000 tasks. The evaluation of scheduling algorithms' performance for WSCC problems has extensively employed the aforementioned workflows, which we consequently have incorporated into our experiments. Table 3 presents the DAG characteristics of these workflows, encompassing node and edge counts, as well as the average task runtime on a reference compute unit.

The main difference between the existing benchmark workflows impacts task length and complexity since they are taken from different projects and different scientific contexts. For instance, the Montage workflow was created from the tool developed by NASA/IPAC to generate custom mosaics of the sky using input images, whereas the Cybershake workflow is used by the Southern California Earthquake Center for the characterization of hazards caused by earthquakes in a region using the probabilistic seismic hazard analysis technique (Bharathi et al., 2008). More information regarding the aforementioned workflows can be found in the study by Juve et al. (2013). An example of the Cybershake small workflow is shown in Fig. 5 in the form of a DAG.

5.2. Instances

Amazon Elastic Computing Cloud (EC2) stands as a prominent Infrastructure as a Service (IaaS) platform, offering users access to Amazon's expansive computing infrastructure. One of the key features of EC2 is the ability for users to dynamically adjust their infrastructure,

[1] https://pegasus.isi.edu/workflow_gallery/

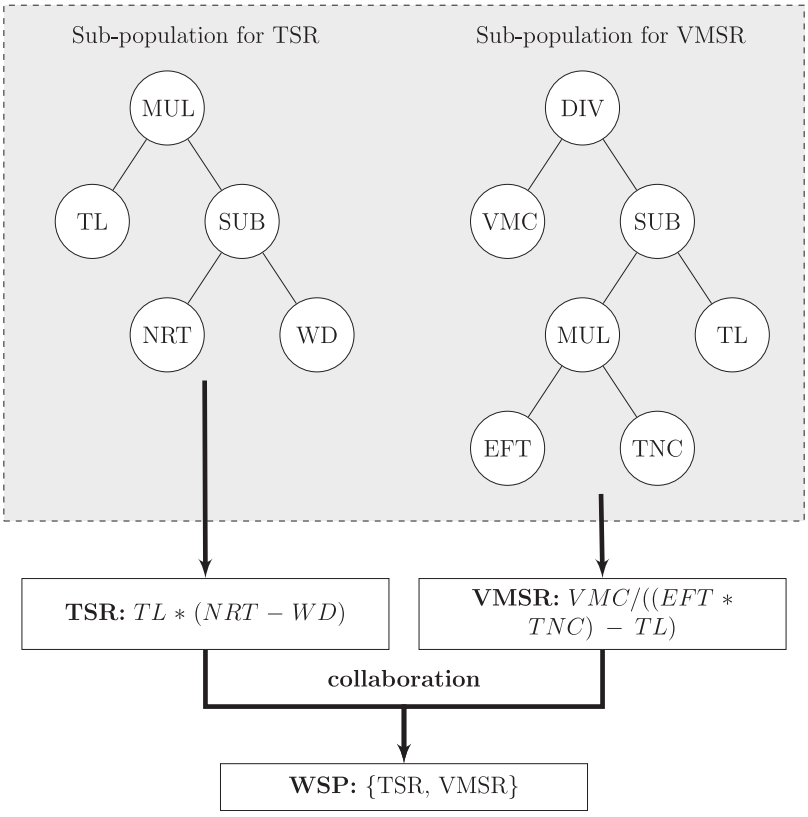


Fig. 4. Collaboration scheme of two sub-populations.

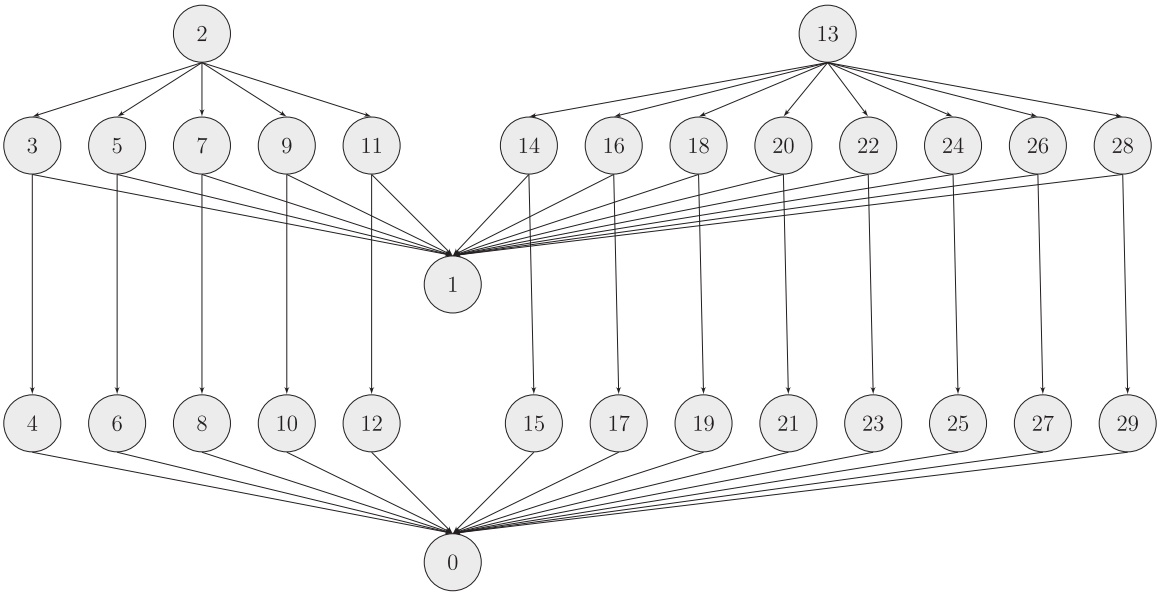


Fig. 5. An example DAG of the Cybershake small workflow.

expanding or shrinking as needed through the acquisition or release of instances. For our experimental setup, we specifically opted for EC2 On-Demand Instances,[2] which enable users to pay solely for the utilized resources and the duration of computation, with charges applied on an hourly basis. These instances come in various types, each offering distinct performance characteristics and price points. To construct a heterogeneous environment, we selected four instance types, each possessing unique attributes. The performance of these instances is typically measured in elastic compute units, with equivalence to a 1.0–1.2 GHz 2007 Opteron or 2007 Xeon processor. It is noteworthy that WorkflowSim, the simulation engine chosen for this study, requires the machines' performance in million instructions *per* second (MIPS).

[2] https://aws.amazon.com/ec2/instance-types/

Table 3
Benchmark workflow characteristics.

Workflow	Size	Number of tasks	Number of edges	Average runtime in sec[a]
Cybershake	small	30	112	23.77
	medium	50	188	29.32
	large	100	380	31.53
Epigenomics	small	24	75	681.54
	medium	46	148	844.93
	large	100	322	3954.90
Inspiral	small	30	95	206.78
	medium	50	160	226.19
	large	100	319	206.12
Montage	small	25	95	8.44
	medium	50	206	9.78
	large	100	433	10.58
Sipht	small	30	91	178.92
	medium	60	198	194.48
	large	100	335	175.55

[a] Based on a reference compute unit (Xeon@2.33 GHz CPUs)

Table 4
VM configurations of Amazon EC2 instances.

	t1.micro	m1.small	m1.medium	m1.large
CPU (in MIPS)	478	1026	2070	2644
RAM (in MB)	613	1700	3750	7500
Cost (in $/hour)	0.0125	0.025	0.0499	0.0998

Consequently, an existing computational benchmark analysis[3] was consulted to express the machines' performance in the requisite MIPS format. Table 4 summarizes the characteristics of the selected VMs for our computational experiments.

5.3. Genetic programming configuration

For the GPHH algorithm, we systematically explore various configurations in exhaustive preliminary experiments. The most promising settings are summarized n Table 5.

A population of 200 individuals evolving over 50 generations indicated a good balance between exploration and exploitation. The initial population is generated using the ramped half-and-half method with a depth ranging between 2 and 6, facilitating a diverse and balanced initialization. Throughout the evolutionary procedure, individuals undergo reproduction via crossover and mutation with probabilities of 90% and 10%, respectively. To regulate the size of the evolved trees and to prevent excessive growth, we set a maximum depth of 17 and 7 for crossover and mutation operators, respectively.

In the context of the CC, a noteworthy aspect is the synergistic collaboration among individuals from distinct sub-populations (Shi and Gao, 2017). Specifically, in addressing the optimization challenges of WSCC problems, our proposed integrated scheduling algorithm entails the combination of individuals from two sub-populations: TSR and VMSR. This fusion is imperative for the formulation of a comprehensive WSP. The collaboration strategies for this integration can be classified into two categories (Wiegand et al., 2001). The first category encompasses schemes rooted in randomness, with the simplest approach involving the random shuffling of both sub-populations (Panait et al., 2006). At its core, the random shuffling mechanism entails the unbiased mixing of individuals from both sub-populations, which ensures a diverse pool of candidate solutions for the WSP. This mixing is achieved by randomly selecting individuals from each sub-population and combining them to form new solutions. This process is iteratively repeated, which allows the exploration of a wide range of potential solutions. One notable advantage of the random shuffling collaboration scheme lies in its ability to prevent premature convergence and promote exploration within the solution space. By introducing randomness into the integration process, the algorithm avoids being trapped in local optima and encourages the discovery of novel, high-quality solutions. This stochastic nature enables an effective navigation of complex optimization landscapes. Furthermore, the random shuffling collaboration scheme exhibits computational efficiency by leveraging the evaluation of individuals from both sub-populations only once during the integration process. This streamlined approach minimizes computational overhead while maximizing the diversity of generated solutions, thereby enhancing the algorithm's scalability and efficacy. To this end, the random shuffling collaboration scheme was chosen for our experimental study.

Table 5
MO-GPHH settings.

Parameter	Value
Number of Generations	50
Population size	200
Initial population	Ramped half-and-half (depth 2–6)
Crossover probability	90%
Maximum depth for crossover	17
Mutation rate	10%
Maximum depth for mutation	7
Collaboration scheme[a]	Random shuffling
Training set size	10
Validation set size	30

[a] Only applicable to MO-GPHH$_{COEV}$.

Another crucial consideration pertains to the size of the training and validation sets. We propose the random generation of 10 datasets for the training set and 30 for the validation set, with each dataset comprising 5 randomly selected workflows from the benchmark workflow list (refer to Table 3). This approach ensures diversity for the training of robust priority rules for solving unseen instances of the addressed WSCC problem. In the evaluation phase, the simulation environment incorporates a total of 40 VMs, distributed as 10 of each type (refer to Table 4). This configuration aims to provide a realistic and comprehensive computing environment for assessing the performance of evolved individuals.

5.4. Benchmark algorithms

As mentioned in the introductions, constructive priority rule-based heuristics are the most commonly used solution method in WSCC due to their fast execution and straightforward implementation. Moreover, the GPHH approach proposed in this study operates on the automatically evolving of new priority rules. Therefore, the proposed solution will be compared to the following selection of the most frequently applied rule-based heuristics in the WSCC literature:

- *First-Come-First-Served (FCFS)*: tasks are scheduled by order of arrival (Rimal and Maier, 2017). This is the simple version of the algorithm used in schedulers such as the Hadoop Fair Scheduler (Polato et al., 2014).
- *Max-Min*: assigns task with the highest execution time to the machine with a minimum completion time (Devipriya and Ramesh, 2013).
- *Min-Min*: assigns tasks with the lowest execution time to machines with a minimum completion time (Patel et al., 2015).
- *Minimum Completion Time (MCT)*: selects tasks based on their minimum expected completion time (Rimal and Maier, 2017).
- *Round Robin (RR)*: selects the first available VM to assign to a task (Pradhan et al., 2016).

[3] https://www.servethehome.com/testing-linux-benchmark-suite-amazon-ec2-instances/

– *Heterogeneous Earliest Finish Time (HEFT)*: schedules workflows on a distributed system with heterogeneous VMs based on their estimated finish times (Topcuoglu et al., 2002).

5.5. Performance metrics

With the introduction of the problem done in the background section, this work will be dedicated to minimizing the makespan and costs. Therefore, as we are dealing with an MO problem, it will be difficult or even impossible to find an overall dominating solution without trade-offs in some of the objectives, therefore the concept of hypervolume (HV) is introduced to measure the performance with respect to multiple objectives (see, e.g., Zitzler et al., 2003). HV is one of the most common indicators for evaluating algorithms' performance in evolutionary MO optimization. This indicator measures the size of the objective space that is dominated by the non-dominated solutions. Larger HV values imply a better evaluation of the rated algorithm. Furthermore, the HV can be normalized, resulting in the so-called hypervolume ratio (HVR) (see, e.g., Van Veldhuizen and Lamont, 1999). To calculate this value, first, a reference HV (HV_{ref}) must be determined. The HV_{ref} is typically defined as the actual Pareto front. Since the exact solutions are not known to the problem at hand, the HV_{ref} is proposed to be derived from all solutions obtained during the entire experiment. We then apply the following formula to calculate the HVR:

$$HVR = \frac{HV}{HV_{Ref}} \tag{15}$$

6. Experimental results

This section is dedicated to the experimental results of this study. First, the three proposed MO-GPHH algorithms regarding task scheduling, VM selection, and the COEV approach will be validated and discussed. The three alternatives will be denoted as MO-GPHH$_{TSR}$, MO-GPHH$_{VMSR}$, and MO-GPHH$_{COEV}$. In the second part of this section, the performance of the best candidates will be compared to the benchmark algorithms from the literature. The third part is dedicated to the complexity analysis of the evolved rules. Then, the robustness of the best-evolved priority rules will be evaluated by validating them on a set of unseen instances. Finally, the most relevant achievements are summarized and discussed. For all experiments, we validate the significance between all candidates' results with a Wilcoxon signed-rank test with a significance level of 5% (Wilcoxon, 1992)

6.1. MO-GPHH experiment

To evaluate the performance of the three proposed MO-GPHH algorithms, each candidate is executed for 10 independent runs in order to ensure statistically representative results. Table 6 shows the performance of the proposed algorithms in the form of HVR. On the one side, the benchmark heuristics reached an HVR of 57.57%. On the other side, the three proposed candidate algorithms showed a different outcome, varying from an HVR of 18.50% to 99.55%, which is equivalent to a 67.87% decrease and 72.91% improvement compared to the benchmark HVR. While the effectiveness of the MO-GPHH$_{TSR}$ was rather limited, the MO-GPHH$_{VMSR}$ and MO-GPHH$_{COEV}$ alternatives were able to evolve heuristics that significantly outperformed the benchmark heuristics regarding the HVR. Although the difference between the performance of the latter two candidates is small, MO-GPHH$_{COEV}$ achieved a significantly larger HVR, revealing itself as the overall best algorithm of the comparison. Furthermore, MO-GPHH$_{COEV}$ achieved the most consistent result over the 10 independent runs compared to its competitors. More details regarding the convergence and the HVR of all 10 runs are shown in Fig. 6.

Fig. 7 shows the individuals of the final populations of the three candidate algorithms for a representative run projected into the objective space. It can be seen that the MO-GPHH$_{TSR}$ operates on a limited solution space. A comparison to the MO-GPHH$_{VMSR}$ leads to the conclusion that the task scheduling decision is much less impactful to the overall performance than the VM scheduling decision within the proposed constructive procedure. Nevertheless, by considering both decisions simultaneously by coevolving TSR and VMSR, the performance of the MO-GPHH$_{COEV}$ showed a small improvement with respect to the HVR achieved by the MO-GPHH$_{VMSR}$ algorithm. Most individuals of the final population of the MO-GPHH$_{VMSR}$ are dominated by the individuals generated by the MO-GPHH$_{COEV}$ algorithm. In particular, the performance regarding costs could be slightly enhanced. Moreover, Fig. 7 indicates that all benchmark heuristics are dominated by rules evolved by either the MO-GPHH$_{VMSR}$ or the MO-GPHH$_{COEV}$ algorithm, revealing their effectiveness in automatically evolving non-dominated heuristics superior to existing ones designed by humans.

Table 6
HVR Values for the three candidate algorithms compared to the benchmark heuristics (in percentage, the best values are shown in bold).

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$
mean	18.50 (↓67.87%)	99.09 (↑72.12%)	**99.55 (↑72.91%)**
max	19.71 (↓65.76%)	99.30 (↑72.48%)	**99.69 (↑73.15%)**
min	16.83 (↓70.76%)	98.73 (↑71.50%)	**99.42 (↑72.69%)**
standard deviation	0.81	0.20	**0.08**

Table 7
HVR of the evolved rules on a set of validation instances compared to the benchmark heuristics (in percentage, the best values are shown in bold).

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$
mean	23.98 (↓53.85%)	97.21 (↑87.12%)	**98.85 (↑90.27%)**
max	26.03 (↓49.89%)	98.17 (↑88.97%)	**99.09 (↑90.74%)**
min	22.11 (↓57.43%)	95.61 (↑84.04%)	**98.49 (↑89.57%)**
standard deviation	1.13	0.72	**0.23**

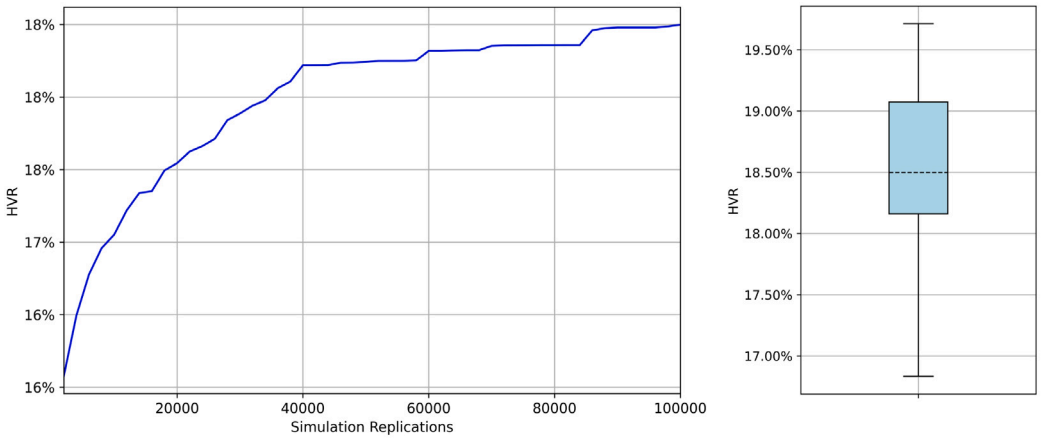
6.2. Robustness analysis

In the course of our robustness analysis, we assess the efficacy of the evolved rules on a set of unseen instances. Specifically, following the genetic programming configuration outlined in Section 5.3, the evolved rules undergo a validation on a collection of 30 randomly generated instances (employing the same settings for generation as for the training phase but with different random seeds). The outcomes of this validation process are then compared to the performance of the benchmark heuristics.

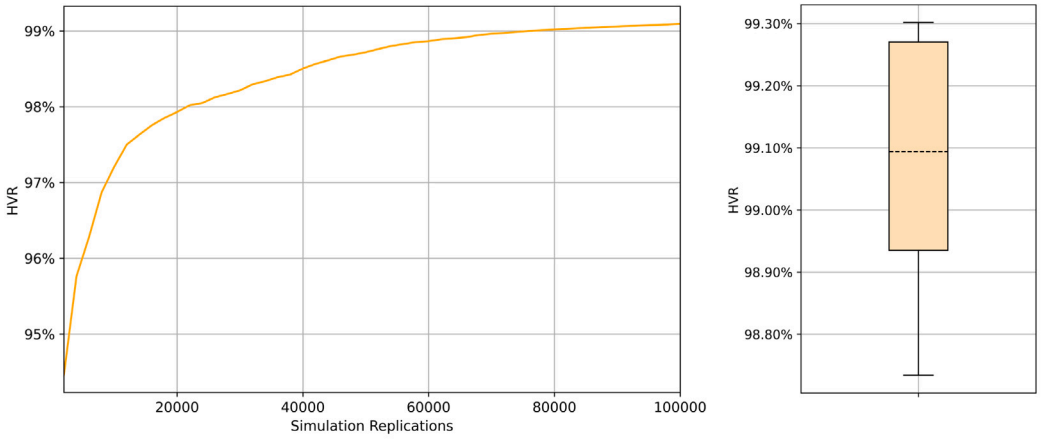
The benchmark algorithms exhibit an HVR of 51.95%. Consistent with the findings on the training instances, the MO-GPHH$_{TSR}$ algorithm resulted in an average decrease of 54.18%, yielding an HVR of 23.80%. Conversely, the evolved rules generated by the MO-GPHH$_{VMSR}$ algorithm demonstrated a significant improvement, enhancing the HVR by 87.11% to a final value of 97.21%. The most substantial enhancement, however, was observed in the case of the MO-GPHH$_{COEV}$ algorithm, where the coevolution of both priority rules led to a remarkable 90.26% improvement of the benchmark heuristics' performance, resulting in an average HVR of 98.85%. Another noteworthy observation concludes that the evolved rules by all three approaches are presently more robust to new situations than the benchmark heuristics, as the improvements are significantly larger compared to the results on the training instances (see Table 7 for detailed HVR results).

6.3. Evolution and execution time

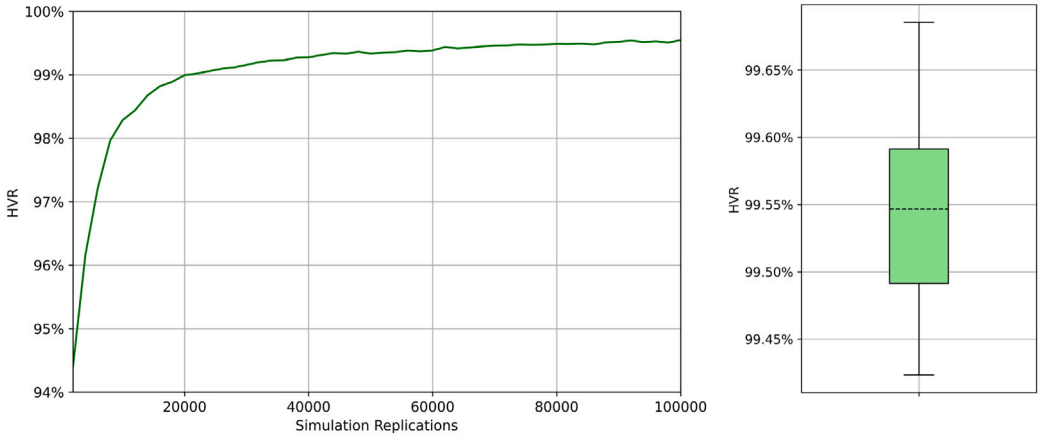
In Table 8, the average evolution time *per* candidate and run is presented. While the training time may be less critical than the execution time in practical applications, a comprehensive analysis is undertaken for completeness. Notably, the benchmark heuristics require no training time, as they are pre-established by experts. Therefore, a comparison



(a) MO-GPHH$_{\mathrm{TSR}}$.



(b) MO-GPHH$_{\mathrm{VMSR}}$.



(c) MO-GPHH$_{\mathrm{COEV}}$.

Fig. 6. Convergence curves (left) and boxplots (right) of the HVR achieved by the three candidate algorithms.

to the benchmark heuristics is not possible for this evaluation. MO-GPHH$_{\mathrm{TSR}}$ takes an average of approximately 348 min for evolving TSRs, while MO-GPHH$_{\mathrm{VMSR}}$ demands about 1.5 times more computational time for evolving VMSRs. The coevolution of TSRs and VMSRs via MO-GPHH$_{\mathrm{COEV}}$ requires nearly 3.5 times the evolution time of MO-GPHH$_{\mathrm{TSR}}$, highlighting the coevolutionary approach's significantly more demanding evolutionary procedure.

Despite similar numbers of fitness evaluations for all three algorithms (the computational most demanding part of the algorithms), differences emerge in total average training time. This dissimilarity

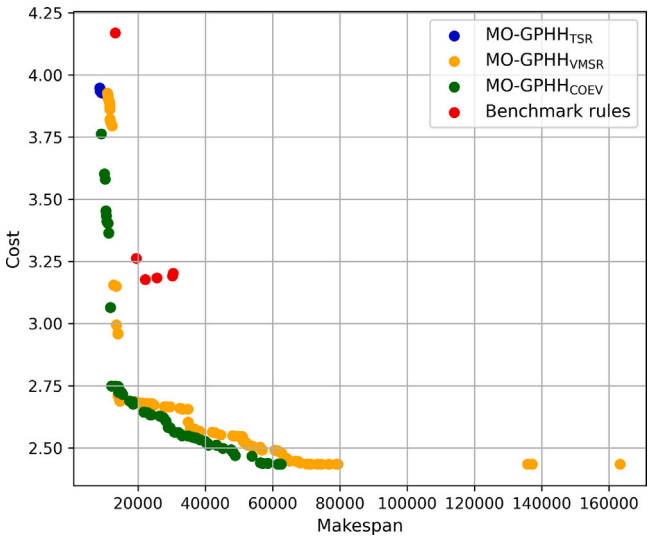


Fig. 7. Final populations of a representative run (run 4) of the three candidate algorithms compared to the benchmark heuristics.

Table 8
Evolution time for the three candidate algorithms (in minutes, the best values are shown in bold).

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$
mean	**348.85**	525.92	1185.85
max	**356.58**	625.87	1450.40
min	**340.98**	413.35	928.90
standard deviation	**5.75**	74.86	156.97

Table 9
Number of non-dominated individuals evolved by the three candidate algorithms (the best values are shown in bold).

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$
mean	9	88	**100**
max	15	97	**116**
min	2	78	**82**
standard deviation	4	7	**13**

is attributed to the number of non-dominated rules identified during evolution, with MO-GPHH$_{VMSR}$ and MO-GPHH$_{COEV}$ finding approximately 10 and 11 times more non-dominated rules than MO-GPHH$_{TSR}$ (see Table 9). Furthermore, given the dual sub-population structure, MO-GPHH$_{COEV}$ requires evolutionary operations such as crossover and mutation for twice the number of individuals. Additional collaborative operations cause a further increase in computational time for coevolving heuristics for the WSCC problem.

The average execution time *per* candidate algorithm is detailed in Table 10, providing insights into the computational efficiency of each approach. The benchmark heuristics demonstrate an average construction time of 0.37 s for generating workflow schedules. On the one hand, TSRs evolved by the MO-GPHH$_{TSR}$ algorithm require, on average, 3.29% longer to generate schedules compared to the benchmark heuristics. On the other hand, VMSRs evolved by MO-GPHH$_{VMSR}$ algorithm incur an average increase of 47.74% in execution time. These discrepancies to the benchmark rules can be attributed to the increasing complexity of the evolved rules, which may grow to a maximum tree depth of 17 (see Table 5). As the priority rules become more complex, the computational time required for calculating task and VM priority values intensifies. Based on our observations, we can conclude that operating on the VMSR has a bigger impact on the overall efficiency of the algorithm than optimizing on the TSR. Notably, the MO-GPHH$_{COEV}$ algorithm, involving the coevolution of two rules, incurs a substantial

Table 10
Execution time for the three candidate algorithms compared to the benchmark heuristics (in seconds, the best values are shown in bold).

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$
mean	**0.38** (↑**3.29**%)	0.55 (↑47.74%)	0.67 (↑80.16%)
max	**0.39** (↓**6.09**%)	0.83 (↑101.26%)	0.92 (↑124.49%)
min	**0.38** (↑**7.91**%)	0.41 (↑16.33%)	0.47 (↑32.97%)
standard deviation	**0.002**	0.092	0.117

Table 11
Number of terminal nodes *per* evolved rule.

	MO – GPHH$_{TSR}$	MO – GPHH$_{VMSR}$	MO – GPHH$_{COEV}$	
	TSR	VMSR	TSR	VMSR
mean	26.50	46.94	20.61	67.16
max	70	148	95	179
min	5	4	1	6
standard deviation	15.24	26.37	21.45	32.78

relative increase in execution time compared to the competing heuristics, given the deepened complexity of both TSR and VMSR. Schedules constructed by rules evolved through MO-GPHH$_{COEV}$ necessitate an average of 0.67 s, 80.16% more time than required by the benchmark heuristics. Despite the MO-GPHH$_{COEV}$ algorithm indicating the slowest solution, its absolute solving time remains relatively low and indicates with less than a second still its practicability for adapting to dynamic real-world CC scenarios.

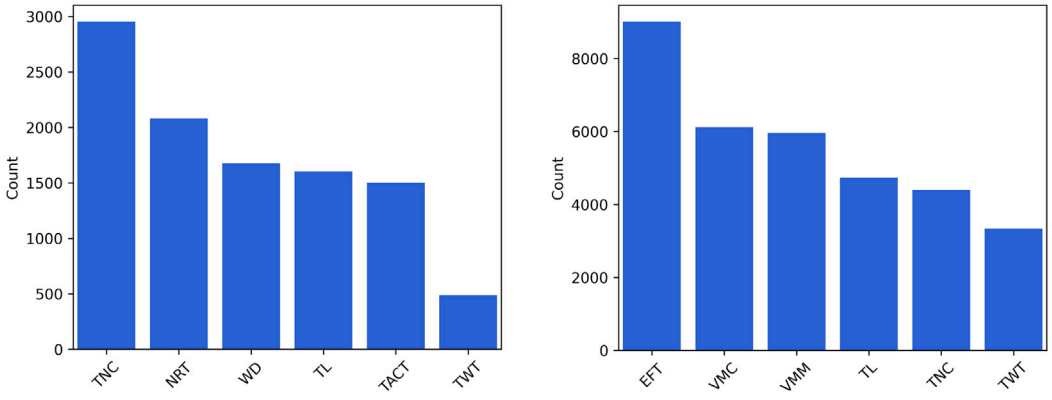
Our proposed algorithm utilizes GP to autonomously learn new priority rules for efficiently solve WSCC problems. While the computational demand for training new rules increases with larger problem sizes, this process can be conducted offline. Once evolved, these rules can be readily applied to quickly schedule workflows even in large-scale cloud environments. This approach ensures scalability by leveraging pre-trained rules to expedite scheduling processes.

6.4. Complexity analysis

In this section, we analyze the complexity and characteristics of the evolved rules in our proposed method. The complexity of these rules is assessed by measuring their size, specifically the number of terminal nodes. Additionally, we investigate the composition of the rules by counting the frequency of each terminal node used in the final generation's population.

To quantify the complexity of the evolved rules, we first measure the size of each rule by counting the number of terminal nodes. As presented in Table 11, the complexity of the evolved VMSRs is generally higher than that of the TSRs. This increased complexity can be attributed to the larger objective space that the VMSR is able to explore, as illustrated in Fig. 7. When both types of rules are coevolved, the overall size of the rules tends to be slightly larger. This increased rule size also corresponds to the execution times shown in Table 10, where more complex rules require more computational effort to execute, thus slowing down the entire algorithm. However, these more complex rules performed better overall in terms of optimizing the makespan and costs.

To further analyze the composition of the evolved rules, we count the appearances of each terminal node within the coevolved rules (see Fig. 8). This analysis reveals differences in the significance of various terminal nodes between TSR and VMSR. For the TSR, TNC emerges as the most significant factor, indicated by its frequent selection during the evolution of these rules. In contrast, for the VMSR, EFT is the most impactful factor. This suggests that EFT plays a critical role in the performance of the VMSR, while TNC is much less relevant. Interestingly, for both TSR and VMSR, TWT is among the least frequently selected terminals, indicating its relatively minor importance in the evolution of both types of rules.



(a) TSR. (b) VMSR.

Fig. 8. Total count of terminals of the evolved rules by the MO-GPHH$_{COEV}$ algorithm.

6.5. Discussion

Given the dual nature of scheduling decisions involving task scheduling and VM allocation, three distinct approaches were proposed. Initially, each decision's evolution process was explored in isolation, aligning with prevailing approaches in MO-GPHH-based WSCC literature. Subsequently, we proposed a cooperative coevolutionary procedure to concurrently address both priority rules, marking the primary innovation of this work. Our findings revealed that within the proposed cloud environment, VM allocation significantly outweighs task scheduling in comprehensively exploring the objective space. While operating on the task scheduling decision showed good improvements in makespan, VM allocation proved more effective in minimizing costs. Computational experiments underscored the coevolutionary method's superiority over formerly isolated approaches, particularly in terms of the HVR. Notably, the coevolutionary strategy demonstrated a more comprehensive exploration of the objective space in both makespan improvement and cost minimization. Our approach successfully improved the HVR up to 72.91% compared to benchmark heuristics.

Another remarkable finding showcases the robustness of evolved rules in new scenarios. Validation on unseen instances demonstrated the cooperative coevolutionary approach's capacity to generate more resilient rules compared to alternative candidates, including benchmark heuristics. Although the coevolutionary process exhibited a higher computational demand, the marginal difference in execution time for generating solutions, a fraction of a second within the test environment, renders the proposed method highly practical for dynamic real-world scenarios.

The implementation of our work encountered limitations related to the system architecture. The use of the WorkflowSim framework, while fundamental to our study, is constrained by its lack of support since 2015. Therefore, an accurate representation of modern IaaS architectures in cloud environment simulations is hindered by limited possibilities for the cloud environment's configuration. To overcome this, future work should extend our approach by incorporating a workflow parser for CloudSimPlus, a Java CC simulation framework offering ongoing development and enhanced options for CC simulations, enabling the exploration of alternative scenarios and more realistic configurations such as dynamically arriving workflows.

7. Conclusion and future direction

This study introduces a novel simulation-based MO-GPHH approach designed to tackle complex WSCC challenges. By leveraging evolutionary computation and machine learning to evolve novel heuristics, our method effectively minimizes makespan and costs in workflow scheduling. Through the proposed cooperative coevolutionary procedure, which concurrently addresses task scheduling and VM allocation decisions, our approach outperforms formerly isolated approaches, demonstrating significant improvements in HVR. Although encountering limitations related to system architecture, the proposed method exhibits practicality for real-world scenarios.

Despite promising results, several avenues for future research warrant exploration. Fine-tuning GP parameters, such as population size and the design of the training set, could provide valuable insights into enhancing evolved rules' performance and robustness. A comprehensive analysis aims to determine the optimal number of candidate solutions *per* generation that balances computational efficiency with the ability to explore a diverse solution space effectively. Careful consideration in designing the training set involves evaluating its representativeness and diversity to ensure the evolved heuristics generalize well across scenarios. Fine-tuning these parameters is essential for enhancing the overall performance and robustness of evolved rules.

Although the evolution of new heuristics can be achieved automatically and offline, the exploration of methods to replace computationally intense simulation-based fitness evaluations can be identified as another noteworthy research direction. The adoption of surrogate-assisted fitness evaluations holds promise in reducing evolutionary time by predicting fitness for newly generated rules through machine learning, potentially using regression models to replace expensive simulation replications.

While our method produces a set of non-dominated heuristics for the DM, another logical extension represents the incorporation of the DM's preferences into the exploration of the objective space. Such an interactive approach could substantially support the guidance of the DM toward the most appealing solution, if any, thereby facilitating the decision-making process of finding an appropriate compromise between competing objectives.

CRediT authorship contribution statement

Tomás Zaki: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Yannik Zeiträg:** Writing – review & editing, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Rui Neves:** Writing – review & editing, Supervision, Methodology, Conceptualization. **José Rui Figueira:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Data availability

Data will be made available on request.

Acknowledgments

This work is funded by national funds through FCT (Fundação para a Ciência e a Tecnologia) and MCTES (Ministério da Ciência, Tecnologia e Ensino Superior), and when applicable co-funded by EU funds under the project UIDB/EEA/50008/2020. Yannik Zeiträg acknowledges the support by national funds through Fundação para a Ciência e a Tecnologia, reference: BD/05314/2021. Yannik Zeiträg and José Rui Figueira acknowledge the Portuguese national funds through the FCT - Foundation for Science and Technology, I.P., under the project UIDB/00097/2020.

References

Abd Elaziz, M., Xiong, S., Jayasena, K., Li, L., 2019. Task scheduling in cloud computing based on hybrid moth search algorithm and differential evolution. Knowl.-Based Syst. 169, 39–52. http://dx.doi.org/10.1016/j.knosys.2019.01.023.

Alkhanak, E.N., Lee, S.P., 2018. A hyper-heuristic cost optimisation approach for scientific workflow scheduling in cloud computing. Future Gener. Comput. Syst. 86, 480–506. http://dx.doi.org/10.1016/j.future.2018.03.055.

Alsaidy, S.A., Abbood, A.D., Sahib, M.A., 2022. Heuristic initialization of PSO task scheduling algorithm in cloud computing. J. King Saud Univ.-Comput. Inf. Sci. 34 (6), 2370–2382. http://dx.doi.org/10.1016/j.jksuci.2020.11.002.

Arabnejad, H., Barbosa, J.G., 2013. List scheduling algorithm for heterogeneous systems by an optimistic cost table. IEEE Trans. Parallel Distrib. Syst. 25 (3), 682–694. http://dx.doi.org/10.1109/TPDS.2013.57.

Bharathi, S., Chervenak, A., Deelman, E., Mehta, G., Su, M.-H., Vahi, K., 2008. Characterization of scientific workflows. In: 2008 Third Workshop on Workflows in Support of Large-Scale Science. IEEE, pp. 1–10. http://dx.doi.org/10.1109/WORKS.2008.4723958.

Bhardwaj, A.K., Gajpal, Y., Surti, C., Gill, S.S., 2020. HEART: Unrelated parallel machines problem with precedence constraints for task scheduling in cloud computing using heuristic and meta-heuristic algorithms. Softw. - Pract. Exp. 50 (12), 2231–2251. http://dx.doi.org/10.1002/spe.2890.

Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M., 2015. Automated design of production scheduling heuristics: A review. IEEE Trans. Evol. Comput. 20 (1), 110–124. http://dx.doi.org/10.1109/TEVC.2015.2429314.

Burke, E., Gustafson, S., Kendall, G., 2003a. Ramped half-n-half initialisation bias in GP. In: Genetic and Evolutionary Computation Conference. Springer, pp. 1800–1801. http://dx.doi.org/10.1007/3-540-45110-2_71.

Burke, E.K., Hyde, M.R., Kendall, G., Ochoa, G., Ozcan, E., Woodward, J.R., 2009. Exploring hyper-heuristic methodologies with genetic programming. In: Computational Intelligence. Springer, pp. 177–201. http://dx.doi.org/10.1007/978-3-642-01799-5_6.

Burke, E.K., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., Woodward, J.R., 2010. A classification of hyper-heuristic approaches. In: Handbook of Metaheuristics. Springer, pp. 449–468. http://dx.doi.org/10.1007/978-1-4419-1665-5_15.

Burke, E., Kendall, G., Newall, J., Hart, E., Ross, P., Schulenburg, S., 2003b. Hyper-heuristics: An emerging direction in modern search technology. In: Glover, F., Kochenberger, G.A. (Eds.), Handbook of Metaheuristics. In: International Series in Operations Research & Management Science, vol. 57, Kluwer Academic Publishers, Boston and London, ISBN: 978-1-4020-7263-5, pp. 457–474. http://dx.doi.org/10.1007/0-306-48056-5_16.

Calheiros, R., Ranjan, R., De Rose, C., Buyya, R., 2009. CloudSim: A novel framework for modeling and simulation of cloud computing infrastructures and services.

Chen, W., Deelman, E., 2012. Workflowsim: A toolkit for simulating scientific workflows in distributed environments. In: 2012 IEEE 8th International Conference on E-Science. IEEE, pp. 1–8. http://dx.doi.org/10.1109/eScience.2012.6404430.

Chen, H., Ding, G., Qin, S., Zhang, J., 2021. A hyper-heuristic based ensemble genetic programming approach for stochastic resource constrained project scheduling problem. Expert Syst. Appl. 167, 114174. http://dx.doi.org/10.1016/j.eswa.2020.114174.

Chiang, M.-L., Hsieh, H.-C., Cheng, Y.-H., Lin, W.-L., Zeng, B.-H., 2023. Improvement of tasks scheduling algorithm based on load balancing candidate method under cloud computing environment. Expert Syst. Appl. 212, 118714. http://dx.doi.org/10.1016/j.eswa.2022.118714.

Deb, K., Pratap, A., Agarwal, S., Meyarivan, T., 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans. Evol. Comput. 6 (2), 182–197. http://dx.doi.org/10.1109/4235.996017.

Devipriya, S., Ramesh, C., 2013. Improved max-min heuristic model for task scheduling in cloud. In: 2013 International Conference on Green Computing, Communication and Conservation of Energy. ICGCE, pp. 883–888. http://dx.doi.org/10.1109/ICGCE.2013.6823559.

Digitale, J.C., Martin, J.N., Glymour, M.M., 2022. Tutorial on directed acyclic graphs. J. Clin. Epidemiol. 142, 264–267. http://dx.doi.org/10.1016/j.jclinepi.2021.08.001.

Dimopoulos, C., Zalzala, A., 2001. Investigating the use of genetic programming for a classic one-machine scheduling problem. Adv. Eng. Softw. (ISSN: 09659978) 32 (6), 489–498. http://dx.doi.org/10.1016/S0965-9978(00)00109-5.

Drake, J.H., Kheiri, A., Özcan, E., Burke, E.K., 2020. Recent advances in selection hyper-heuristics. European J. Oper. Res. 285 (2), 405–428. http://dx.doi.org/10.1016/j.ejor.2019.07.073.

Escott, K.-R., Ma, H., Chen, G., 2020. Genetic programming based hyper heuristic approach for dynamic workflow scheduling in the cloud. In: Database and Expert Systems Applications: 31st International Conference, DEXA 2020, Bratislava, Slovakia, September 14–17, 2020, Proceedings, Part II 31. Springer, pp. 76–90. http://dx.doi.org/10.1007/978-3-030-59051-2_6.

Escott, K.-R., Ma, H., Chen, G., 2023. Cooperative coevolutionary genetic programming hyper-heuristic for budget constrained dynamic multi-workflow scheduling in cloud computing. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). Springer, pp. 146–161. http://dx.doi.org/10.1007/978-3-031-30035-6_10.

Fan, H., Xiong, H., Goh, M., 2021. Genetic programming-based hyper-heuristic approach for solving dynamic job shop scheduling problem with extended technical precedence constraints. Comput. Oper. Res. 134, 105401. http://dx.doi.org/10.1016/j.cor.2021.105401.

Gawali, M.B., Shinde, S.K., 2018. Task scheduling and resource allocation in cloud computing using a heuristic approach. J. Cloud Comput. 7 (1), 1–16. http://dx.doi.org/10.1186/s13677-018-0105-8.

Gil-Gala, F.J., Sierra, M.R., Mencía, C., Varela, R., 2023. Surrogate model for memetic genetic programming with application to the one machine scheduling problem with time-varying capacity. Expert Syst. Appl. 233, 120916. http://dx.doi.org/10.1016/j.eswa.2023.120916.

Hein, F., Almeder, C., Figueira, G., Almada-Lobo, B., 2018. Designing new heuristics for the capacitated lot sizing problem by genetic programming. Comput. Oper. Res. 96, 1–14. http://dx.doi.org/10.1016/j.cor.2018.03.006.

Ho, N., Tay, J.C., 2005. Evolving dispatching rules for solving the flexible job-shop problem. In: The 2005 IEEE Congress on Evolutionary Computation. IEEE, Piscataway, ISBN: 0-7803-9363-5, pp. 2848–2855. http://dx.doi.org/10.1109/CEC.2005.1555052.

Hosseini Shirvani, M., 2023. A survey study on task scheduling schemes for workflow executions in cloud computing environment: classification and challenges. J. Supercomput. 80, 1–54. http://dx.doi.org/10.1007/s11227-023-05806-y.

Hosseini Shirvani, M., Noorian Talouki, R., 2022. Bi-objective scheduling algorithm for scientific workflows on cloud computing platform with makespan and monetary cost minimization approach. Complex Intell. Syst. 8 (2), 1085–1114. http://dx.doi.org/10.1007/s40747-021-00528-1.

Houssein, E.H., Gad, A.G., Wazery, Y.M., Suganthan, P.N., 2021. Task scheduling in cloud computing based on meta-heuristics: review, taxonomy, open challenges, and future trends. Swarm Evol. Comput. 62, 100841. http://dx.doi.org/10.1016/j.swevo.2021.100841.

Imene, L., Sihem, S., Okba, K., Mohamed, B., 2022. A third generation genetic algorithm NSGAIII for task scheduling in cloud computing. J. King Saud Univ.-Comput. Inf. Sci. 34 (9), 7515–7529. http://dx.doi.org/10.1016/j.jksuci.2022.03.017.

Jakobović, D., Jelenković, L., Budin, L., 2007. Genetic programming heuristics for multiple machine scheduling. In: Ebner, M. (Ed.), Genetic Programming. In: Lecture notes in computer science, 0302-9743, vol. 4445, Springer, Berlin, ISBN: 978-3-540-71602-0, pp. 321–330. http://dx.doi.org/10.1007/978-3-540-71605-1_30.

Juve, G., Chervenak, A., Deelman, E., Bharathi, S., Mehta, G., Vahi, K., 2013. Characterizing and profiling scientific workflows. Future Gener. Comput. Syst. 29 (3), 682–692. http://dx.doi.org/10.1016/j.future.2012.08.015.

Kalra, M., Singh, S., 2015. A review of metaheuristic scheduling techniques in cloud computing. Egypt. Inform. J. 16 (3), 275–295. http://dx.doi.org/10.1016/j.eij.2015.07.001.

Koza, J.R., 1992. Genetic programming: On the programming of computers by means of natural selection. In: A Bradford book, MIT Press, Cambridge, Mass., ISBN: 026252791X.

Lin, C.-S., Lin, C.-S., Lin, Y.-S., Hsiung, P.-A., Shih, C., 2013. Multi-objective exploitation of pipeline parallelism using clustering, replication and duplication in embedded multi-core systems. J. Syst. Archit. (ISSN: 1383-7621) 59 (10, Part C), 1083–1094. http://dx.doi.org/10.1016/j.sysarc.2013.05.024, Embedded Systems Software Architecture.

Lin, J., Zhu, L., Gao, K., 2020. A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem. Expert Syst. Appl. 140, 112915. http://dx.doi.org/10.1016/j.eswa.2019.112915.

Luo, J., Vanhoucke, M., Coelho, J., Guo, W., 2022. An efficient genetic programming approach to design priority rules for resource-constrained project scheduling problem. Expert Syst. Appl. 198, 116753. http://dx.doi.org/10.1016/j.eswa.2022.116753.

Madni, S.H.H., Abd Latiff, M.S., Abdullahi, M., Abdulhamid, S.M., Usman, M.J., 2017. Performance comparison of heuristic algorithms for task scheduling in iaas cloud computing environment. PLoS One 12 (5), e0176321. http://dx.doi.org/10.1371/journal.pone.0176321.

Masdari, M., ValiKardan, S., Shahi, Z., Azar, S.I., 2016. Towards workflow scheduling in cloud computing: A comprehensive analysis. J. Netw. Comput. Appl. 66, 64–82. http://dx.doi.org/10.1016/j.jnca.2016.01.018.

Murad, S.A., Muzahid, A.J.M., Azmi, Z.R.M., Hoque, M.I., Kowsher, M., 2022. A review on job scheduling technique in cloud computing and priority rule based intelligent framework. J. King Saud Univ.-Comput. Inf. Sci. http://dx.doi.org/10.1016/j.jksuci.2022.03.027.

Nguyen, S., Mei, Y., Zhang, M., 2017. Genetic programming for production scheduling: A survey with a unified framework. Complex Intell. Syst. 3, 41–66. http://dx.doi.org/10.1007/s40747-017-0036-x.

Nguyen, S., Zhang, M., Johnston, M., Tan, K.C., 2013. Dynamic multi-objective job shop scheduling: A genetic programming approach. In: Etaner-Uyar, A.Ş., Özcan, E., Urquhart, N. (Eds.), Automated Scheduling and Planning. In: Studies in computational intelligence, 1860-949X, vol. 505, Springer, Heidelberg, ISBN: 978-3-642-39303-7, pp. 251–282. http://dx.doi.org/10.1007/978-3-642-39304-4_10.

NoorianTalouki, R., Shirvani, M.H., Motameni, H., 2022. A heuristic-based task scheduling algorithm for scientific workflows in heterogeneous cloud computing platforms. J. King Saud Univ.-Comput. Inf. Sci. 34 (8), 4902–4913. http://dx.doi.org/10.1016/j.jksuci.2021.05.011.

Ozsoydan, F.B., Sağir, M., 2021. Iterated greedy algorithms enhanced by hyper-heuristic based learning for hybrid flexible flowshop scheduling problem with sequence dependent setup times: A case study at a manufacturing plant. Comput. Oper. Res. 125, 105044. http://dx.doi.org/10.1016/j.cor.2020.105044.

Panait, L., Luke, S., Harrison, J.F., 2006. Archive-based cooperative coevolutionary algorithms. In: Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation. pp. 345–352. http://dx.doi.org/10.1145/1143997.1144060.

Patel, G., Mehta, R., Bhoi, U., 2015. Enhanced load balanced min-min algorithm for static meta task scheduling in cloud computing. Procedia Comput. Sci. (ISSN: 1877-0509) 57, 545–553. http://dx.doi.org/10.1016/j.procs.2015.07.385, 3rd International Conference on Recent Trends in Computing 2015 (ICRTC-2015).

Polato, I., Ré, R., Goldman, A., Kon, F., 2014. A comprehensive view of hadoop research—A systematic literature review. J. Netw. Comput. Appl. 46, 1–25. http://dx.doi.org/10.1016/j.jnca.2014.07.022.

Pradhan, P., Behera, P.K., Ray, B., 2016. Modified round robin algorithm for resource allocation in cloud computing. Procedia Comput. Sci. (ISSN: 1877-0509) 85, 878–890. http://dx.doi.org/10.1016/j.procs.2016.05.278, International Conference on Computational Modelling and Security (CMS 2016).

Rajasingh, J.S., Wesley, J.R., 2020. Step into the cloud or stop with virtualization–the project manager's dialectic dilemma. Procedia Comput. Sci. 172, 1077–1083. http://dx.doi.org/10.1016/j.procs.2020.05.157.

Reddy, P.V., Reddy, K.G., 2023. An energy efficient RL based workflow scheduling in cloud computing. Expert Syst. Appl. 234, 121038. http://dx.doi.org/10.1016/j.eswa.2023.121038.

Rimal, B.P., Maier, M., 2017. Workflow scheduling in multi-tenant cloud computing environments. IEEE Trans. Parallel Distrib. Syst. 28 (1), 290–304. http://dx.doi.org/10.1109/TPDS.2016.2556668.

Shi, M., Gao, S., 2017. Reference sharing: A new collaboration model for cooperative coevolution. J. Heuristics 23, 1–30. http://dx.doi.org/10.1007/s10732-016-9322-9.

Shirvani, M.H., Talouki, R.N., 2021. A novel hybrid heuristic-based list scheduling algorithm in heterogeneous cloud computing environment for makespan optimization. Parallel Comput. 108, 102828. http://dx.doi.org/10.1016/j.parco.2021.102828.

Shukri, S.E., Al-Sayyed, R., Hudaib, A., Mirjalili, S., 2021. Enhanced multi-verse optimizer for task scheduling in cloud computing environments. Expert Syst. Appl. 168, 114230. http://dx.doi.org/10.1016/j.eswa.2020.114230.

Singh, H., Tyagi, S., Kumar, P., Gill, S.S., Buyya, R., 2021. Metaheuristics for scheduling of heterogeneous tasks in cloud computing environments: Analysis, performance evaluation, and future directions. Simul. Model. Pract. Theory 111, 102353. http://dx.doi.org/10.1016/j.simpat.2021.102353.

Tay, J.C., Ho, N.B., 2008. Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. Comput. Ind. Eng. (ISSN: 0360-8352) 54 (3), 453–473. http://dx.doi.org/10.1016/j.cie.2007.08.008.

Topcuoglu, H., Hariri, S., Wu, M.-Y., 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE Trans. Parallel Distrib. Syst. 13 (3), 260–274. http://dx.doi.org/10.1109/71.993206.

Tsai, C.-W., Huang, W.-C., Chiang, M.-H., Chiang, M.-C., Yang, C.-S., 2014. A hyper-heuristic scheduling algorithm for cloud. IEEE Trans. Cloud Comput. 2 (2), 236–250. http://dx.doi.org/10.1109/TCC.2014.2315797.

Van Veldhuizen, D.A., Lamont, G.B., 1999. Multiobjective evolutionary algorithm test suites. In: Proceedings of the 1999 ACM Symposium on Applied Computing. pp. 351–357. http://dx.doi.org/10.1145/298151.298382.

Wiegand, R.P., Liles, W.C., De Jong, K.A., et al., 2001. An empirical analysis of collaboration methods in cooperative coevolutionary algorithms. In: Proceedings of the Genetic and Evolutionary Computation Conference. GECCO, vol. 2611, Morgan Kaufmann San Francisco, pp. 1235–1245.

Wilcoxon, F., 1992. Individual comparisons by ranking methods. In: Breakthroughs in Statistics. Springer, pp. 196–202. http://dx.doi.org/10.2307/3001968.

Wu, F., Wu, Q., Tan, Y., 2015. Workflow scheduling in cloud: A survey. J. Supercomput. 71, 3373–3418. http://dx.doi.org/10.1007/s11227-015-1438-4.

Xiao, Q.-z., Zhong, J., Feng, L., Luo, L., Lv, J., 2019. A cooperative coevolution hyper-heuristic framework for workflow scheduling problem. IEEE Trans. Serv. Comput. 15 (1), 150–163. http://dx.doi.org/10.1109/TSC.2019.2923912.

Xie, H., Ding, D., Zhao, L., Kang, K., Liu, Q., 2023. A two-stage preference driven multi-objective evolutionary algorithm for workflow scheduling in the cloud. Expert Syst. Appl. 122009. http://dx.doi.org/10.1016/j.eswa.2023.122009.

Xiong, Y., Huang, S., Wu, M., She, J., Jiang, K., 2017. A Johnson's-rule-based genetic algorithm for two-stage-task scheduling problem in data-centers of cloud computing. IEEE Trans. Cloud Comput. 7 (3), 597–610. http://dx.doi.org/10.1109/TCC.2017.2693187.

Xu, M., Mei, Y., Zhu, S., Zhang, B., Xiang, T., Zhang, F., Zhang, M., 2023. Genetic programming for dynamic workflow scheduling in fog computing. IEEE Trans. Serv. Comput. http://dx.doi.org/10.1109/TSC.2023.3249160.

Yang, Z., Tang, K., Yao, X., 2008. Large scale evolutionary optimization using cooperative coevolution. Inf. Sci. 178 (15), 2985–2999. http://dx.doi.org/10.1016/j.ins.2008.02.017.

Yu, Y., Feng, Y., Ma, H., Chen, A., Wang, C., 2019. Achieving flexible scheduling of heterogeneous workflows in cloud through a genetic programming based approach. In: 2019 IEEE Congress on Evolutionary Computation. CEC, IEEE, pp. 3102–3109. http://dx.doi.org/10.1109/CEC.2019.8789896.

Yu, Y., Ma, H., Chen, G., 2021. Achieving multi-objective scheduling of heterogeneous workflows in cloud through a genetic programming based approach. In: 2021 IEEE Congress on Evolutionary Computation. CEC, IEEE, pp. 1880–1887. http://dx.doi.org/10.1109/CEC45853.2021.9504695.

Yu, Y., Shi, T., Ma, H., Chen, G., 2022. A genetic programming-based hyper-heuristic approach for multi-objective dynamic workflow scheduling in cloud environment. In: 2022 IEEE Congress on Evolutionary Computation. CEC, IEEE, pp. 1–8. http://dx.doi.org/10.1109/CEC55065.2022.9870403.

Zade, B.M.H., Mansouri, N., Javidi, M.M., 2021. SAEA: A security-aware and energy-aware task scheduling strategy by parallel squirrel search algorithm in cloud environment. Expert Syst. Appl. 176, 114915. http://dx.doi.org/10.1016/j.eswa.2021.114915.

Zeiträg, Y., Figueira, J.R., 2023. Automatically evolving preference-based dispatching rules for multi-objective job shop scheduling. J. Sched. 1–26. http://dx.doi.org/10.1007/s10951-023-00783-9.

Zeiträg, Y., Figueira, J.R., Figueira, G., 2024. A cooperative coevolutionary hyper-heuristic approach to solve lot-sizing and job shop scheduling problems using genetic programming. Int. J. Prod. Res. 1–28. http://dx.doi.org/10.1080/00207543.2023.2301044.

Zeiträg, Y., Figueira, J.R., Horta, N., Neves, R., 2022. Surrogate-assisted automatic evolving of dispatching rules for multi-objective dynamic job shop scheduling using genetic programming. Expert Syst. Appl. 209, 118194. http://dx.doi.org/10.1016/j.eswa.2022.118194.

Zhang, J., Cheng, L., Liu, C., Zhao, Z., Mao, Y., 2023. Cost-aware scheduling systems for real-time workflows in cloud: An approach based on genetic algorithm and deep reinforcement learning. Expert Syst. Appl. 234, 120972. http://dx.doi.org/10.1016/j.eswa.2023.120972.

Zhou, Y., Yang, J.-j., Huang, Z., 2020. Automatic design of scheduling policies for dynamic flexible job shop scheduling via surrogate-assisted cooperative co-evolution genetic programming. Int. J. Prod. Res. (ISSN: 0020-7543) 58 (9), 2561–2580. http://dx.doi.org/10.1080/00207543.2019.1620362.

Zhou, Y., Yang, J.-J., Zheng, L.-Y., 2018. Hyper-heuristic coevolution of machine assignment and job sequencing rules for multi-objective dynamic flexible job shop scheduling. IEEE Access 7, 68–88. http://dx.doi.org/10.1109/ACCESS.2018.2883802.

Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., Da Fonseca, V.G., 2003. Performance assessment of multiobjective optimizers: An analysis and review. IEEE Trans. Evol. Comput. 7 (2), 117–132. http://dx.doi.org/10.1109/TEVC.2003.810758.